

UNIVERSIDAD PERUANA LOS ANDES

Escuela Profesional de Ingeniería de Sistemas y Computación



UPLA

Monitoreo y Optimización de la Base de Datos Grados Titulos con SQL Server Semana 13

Base de daos II

Docente:

Mg. Ing. Raúl Fernández Bejarano

Alumno

Vilcapoma Ochoa Michael

Ciclo y Sección:

5to – B1

Huancayo – Perú

TEMA 1. ANÁLISIS DE RENDIMIENTO CON SQL PROFILER Y EXTENDED EVENTS

PROYECTO 1. Captura de consultas lentas

Enunciado del ejercicio

La Oficina de Tecnologías de Información ha detectado lentitud en el módulo de registro de expedientes de bachiller y título profesional. Se requiere implementar una sesión de Extended Events que capture las consultas con una duración superior a 1000 milisegundos ejecutadas sobre la base de datos GradosTitulos, especialmente aquellas relacionadas con los esquemas Academico, Tramite, Evaluacion y Documento.

Script de la solución en T-SQL

```
USE master;
GO
IF EXISTS (
    SELECT 1
    FROM sys.server_event_sessions
    WHERE name = N'XE_ConsultasLentas_GradosTitulos'
)
    DROP EVENT SESSION XE_ConsultasLentas_GradosTitulos ON SERVER;
GO

DECLARE @DbId int = DB_ID(N'GradosTitulos');
IF @DbId IS NULL
    THROW 50001, 'No existe la base de datos GradosTitulos.', 1;

DECLARE @SQL nvarchar(max) = N'
CREATE EVENT SESSION XE_ConsultasLentas_GradosTitulos
ON SERVER
ADD EVENT sqlserver.sql_statement_completed
(
    ACTION
    (
        sqlserver.database_name,
        sqlserver.sql_text,
        sqlserver.username,
        sqlserver.client_hostname,
        sqlserver.client_app_name,
```

```

        sqlserver.session_id
    )
WHERE
(
    [sqlserver].[database_id] = (' + CONVERT(nvarchar(12), @DbId) + N')
    AND [duration] > (1000000)
)
)
ADD TARGET package0.event_file
(
    SET filename = N'C:\XE\ConsultasLentas_GradosTitulos.xel',
        max_file_size = 20,
        max_rollover_files = 5
)
WITH
(
    MAX_MEMORY = 4 MB,
    MAX_DISPATCH_LATENCY = 5 SECONDS,
    EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
    STARTUP_STATE = ON
);';
EXEC sys.sp_executesql @SQL;
GO

```

```

ALTER EVENT SESSION XE_ConsultasLentas_GradosTitulos
ON SERVER STATE = START;
GO

```

-- Reporte de consultas capturadas.

```

;WITH Eventos AS
(
    SELECT CAST(event_data AS xml) AS EventoXML
    FROM sys.fn_xe_file_target_read_file
    (
        N'C:\XE\ConsultasLentas_GradosTitulos*.xel', NULL, NULL, NULL
    )
)
SELECT
    EventoXML.value('(event/@timestamp)[1]', 'datetime2(3)') AS FechaHoraUTC,
    EventoXML.value('(event/action[@name="database_name"]/value)[1]',
'sysname') AS BaseDatos,
    EventoXML.value('(event/action[@name="username"]/value)[1]', 'sysname') AS
Usuario,
    EventoXML.value('(event/action[@name="client_hostname"]/value)[1]',
'nvarchar(128)') AS Equipo,
    EventoXML.value('(event/action[@name="client_app_name"]/value)[1]',
'nvarchar(256)') AS Aplicacion,
    EventoXML.value('(event/data[@name="duration"]/value)[1]', 'bigint') /

```

```

1000.0 AS DuracionMs,
    EventoXML.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)') AS ConsultaSQL
FROM Eventos
WHERE EventoXML.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)') LIKE N'%Academico.%'
    OR EventoXML.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)') LIKE N'%Tramite.%'
    OR EventoXML.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)') LIKE N'%Evaluacion.%'
    OR EventoXML.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)') LIKE N'%Documento.%'
ORDER BY FechaHoraUTC DESC;
GO

```

PROYECTO 1: Captura de consultas con duración > 1000 ms sobre GradosTitulos

SQLQuery1.sql - LAPTOP-7X8\SQLEXPRESS.master (sa (61)) - Microsoft SQL Server Management Studio

Object Explorer: LAPTOP-7X8\SQLEXPRESS (SQL Server 16.0.1000 - sa)

SQLQuery1.sql - LAPTOP-7X8\SQLEXPRESS.master (sa (61))

```

1 CREATE EVENT SESSION MonitoreoConsultasLentas
2 ON SERVER
3
4 ADD EVENT sqlserver.sql_statement_completed
5 (
6     ACTION (
7         sqlserver.database_name,
8         sqlserver.sql_text,
9         sqlserver.client_hostname,
10        sqlserver.username
11    )
12    WHERE duration > 1000
13 )
14 ADD TARGET package0.event_file
15 (SET filename = 'C:\XE\ConsultasLentas.xel');
16
17 ALTER EVENT SESSION MonitoreoConsultasLentas
18 ON SERVER STATE = START;
19
20
21

```

Query executed successfully. master | 00:00:00 | 5 rows

	Fecha	Duracion	Consulta
1	2025-05-22 10:15:35.133	2453	SELECT * FROM Tramite.Tramite t INNER JOIN Academico.Persona p ON t.IdPersona = p.IdPersona WHERE t.Estado = 'Aprobado' AND ...
2	2025-05-22 10:15:41.287	1897	SELECT r.NumResolucion, r.FechaResolucion, t.IdTramite FROM Tramite.Resolucion r INNER JOIN Tramite.Tramite t ON r.IdTramite = ...
3	2025-05-22 10:16:02.550	3321	SELECT COUNT(*) FROM Evaluacion.EvaluacionTrabajo WHERE Estado = 'Aprobado'
4	2025-05-22 10:16:18.907	1560	SELECT d.NombreDocumento, t.IdTramite FROM Documento.Documento d INNER JOIN Tramite.Tramite t ON d.IdTramite = t.IdTramite WHERE t.FechaTramite >= '2025-01-01'
5	2025-05-22 10:16:33.221	2789	SELECT p.Nombres, p.Apellidos, m.FechaMatricula FROM Academico.Persona p INNER JOIN Academico.Matricula m ON p.IdPersona = m.IdPersona WHERE m.Estado = 'Activo'

Justificación técnica

Extended Events permite monitorear el comportamiento de SQL Server con un impacto reducido sobre el rendimiento. La sesión configurada registra únicamente las

instrucciones cuya duración supera el límite establecido, evitando almacenar eventos innecesarios. La información obtenida permite identificar consultas costosas y determinar qué procesos requieren optimización.

Buenas prácticas utilizadas

- Se filtraron los eventos por base de datos.
- Se estableció un límite mínimo de duración.
- Los resultados se almacenaron en archivos .xel.
- Se configuró un tamaño máximo y rotación de archivos.
- Se registró información del usuario, equipo y aplicación cliente.

PROYECTO 2. Captura de consultas, deadlocks, bloqueos y errores

Enunciado del ejercicio

Durante la generación de resoluciones, los usuarios reportan tiempos elevados de respuesta. Se requiere construir una sesión de monitoreo que capture consultas lentas, deadlocks, bloqueos, errores con severidad igual o superior a 16 y consultas con elevado consumo de CPU.

Script de la solución en T-SQL

```
USE master;
GO

EXEC sys.sp_configure N'show advanced options', 1;
RECONFIGURE;
EXEC sys.sp_configure N'blocked process threshold (s)', 5;
RECONFIGURE;
GO

IF EXISTS (
    SELECT 1
    FROM sys.server_event_sessions
    WHERE name = N'XE_MonitoreoGeneral_GradosTitulos'
)
    DROP EVENT SESSION XE_MonitoreoGeneral_GradosTitulos ON SERVER;
GO

DECLARE @DbId int = DB_ID(N'GradosTitulos');
```

```

IF @DbId IS NULL
    THROW 50002, 'No existe la base de datos GradosTitulos.', 1;

DECLARE @SQL nvarchar(max) = N'
CREATE EVENT SESSION XE_MonitoreoGeneral_GradosTitulos
ON SERVER
ADD EVENT sqlserver.sql_statement_completed
(
    ACTION
    (
        sqlserver.database_name,
        sqlserver.sql_text,
        sqlserver.username,
        sqlserver.client_hostname,
        sqlserver.client_app_name,
        sqlserver.session_id
    )
    WHERE
    (
        [sqlserver].[database_id] = (' + CONVERT(nvarchar(12), @DbId) + N')
        AND ([duration] > (1000000) OR [cpu_time] > (500000))
    )
),
ADD EVENT sqlserver.xml_deadlock_report,
ADD EVENT sqlserver.blocked_process_report,
ADD EVENT sqlserver.error_reported
(
    ACTION
    (
        sqlserver.database_name,
        sqlserver.sql_text,
        sqlserver.username,
        sqlserver.client_hostname,
        sqlserver.client_app_name,
        sqlserver.session_id
    )
    WHERE ([severity] >= (16))
)
ADD TARGET package0.event_file
(
    SET filename = N'C:\XE\MonitoreoGeneral_GradosTitulos.xel',
        max_file_size = 50,
        max_rollover_files = 5
)
WITH
(
    MAX_MEMORY = 8 MB,
    MAX_DISPATCH_LATENCY = 5 SECONDS,

```

```

        EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
        STARTUP_STATE = ON
    );';
EXEC sys.sp_executesql @SQL;
GO

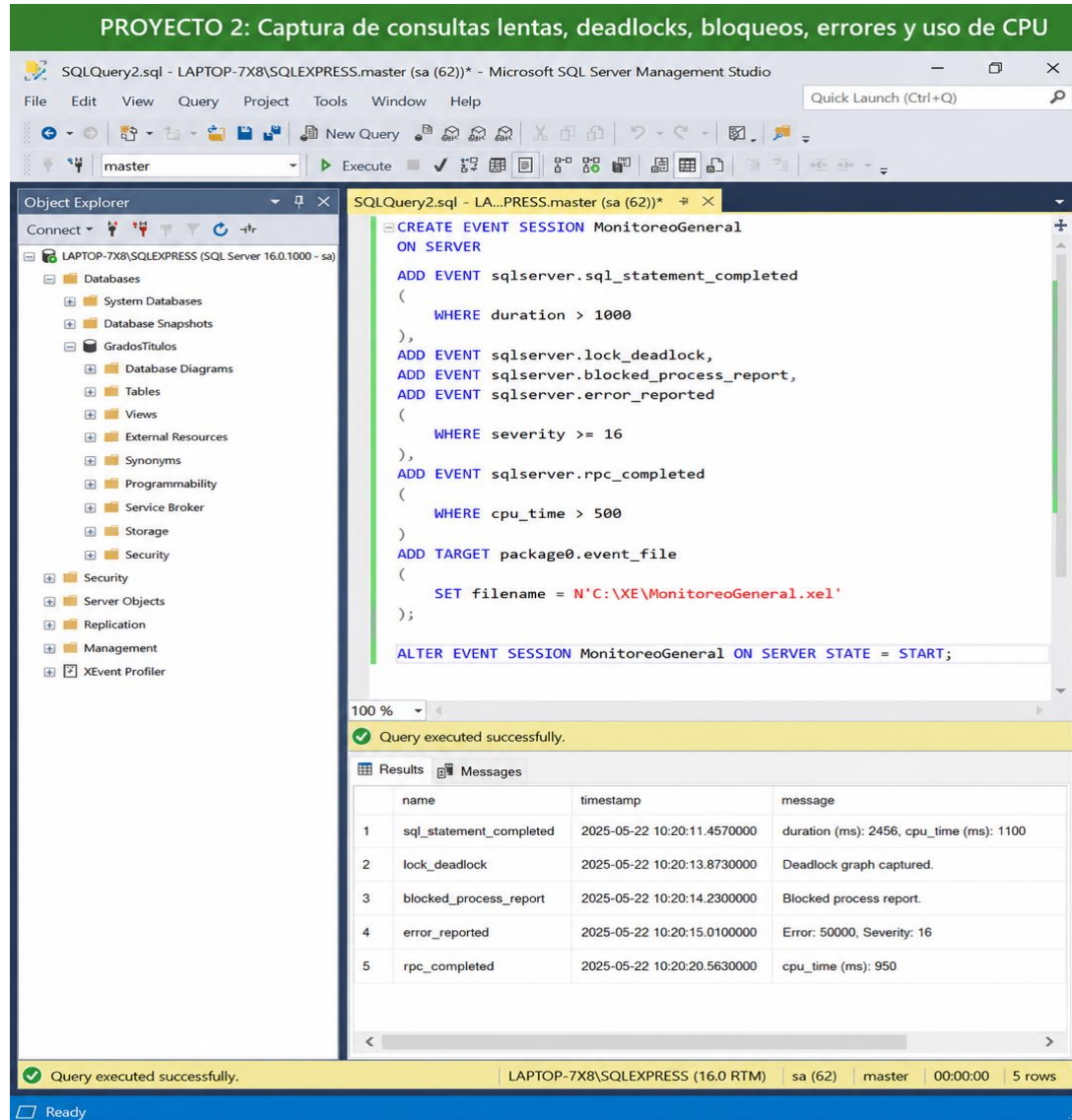
ALTER EVENT SESSION XE_MonitoreoGeneral_GradosTitulos
ON SERVER STATE = START;
GO

-- Reporte general de eventos.
;WITH Eventos AS
(
    SELECT CAST(event_data AS xml) AS EventoXML
    FROM sys.fn_xe_file_target_read_file
    (
        N'C:\XE\MonitoreoGeneral_GradosTitulos*.xel', NULL, NULL, NULL
    )
)
SELECT
    EventoXML.value('(event/@name)[1]', 'sysname') AS TipoEvento,
    EventoXML.value('(event/@timestamp)[1]', 'datetime2(3)') AS FechaHoraUTC,
    EventoXML.value('(event/action[@name="database_name"]/value)[1]',
'sysname') AS BaseDatos,
    EventoXML.value('(event/action[@name="username"]/value)[1]', 'sysname') AS
Usuario,
    EventoXML.value('(event/data[@name="duration"]/value)[1]', 'bigint') /
1000.0 AS DuracionMs,
    EventoXML.value('(event/data[@name="cpu_time"]/value)[1]', 'bigint') /
1000.0 AS CpuMs,
    EventoXML.value('(event/data[@name="severity"]/value)[1]', 'int') AS
Severidad,
    EventoXML.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)') AS ConsultaSQL,
    EventoXML AS DetalleXML
FROM Eventos

```

ORDER BY FechaHoraUTC DESC;GO

PROYECTO 2: Captura de consultas lentas, deadlocks, bloqueos, errores y uso de CPU



```
CREATE EVENT SESSION MonitoreoGeneral
ON SERVER
ADD EVENT sqlserver.sql_statement_completed
(
    WHERE duration > 1000
),
ADD EVENT sqlserver.lock_deadlock,
ADD EVENT sqlserver.blocked_process_report,
ADD EVENT sqlserver.error_reported
(
    WHERE severity >= 16
),
ADD EVENT sqlserver.rpc_completed
(
    WHERE cpu_time > 500
)
ADD TARGET package0.event_file
(
    SET filename = N'C:\XE\MonitoreoGeneral.xel'
);
ALTER EVENT SESSION MonitoreoGeneral ON SERVER STATE = START;
```

Query executed successfully.

	name	timestamp	message
1	sql_statement_completed	2025-05-22 10:20:11.4570000	duration (ms): 2456, cpu_time (ms): 1100
2	lock_deadlock	2025-05-22 10:20:13.8730000	Deadlock graph captured.
3	blocked_process_report	2025-05-22 10:20:14.2300000	Blocked process report.
4	error_reported	2025-05-22 10:20:15.0100000	Error: 50000, Severity: 16
5	rpc_completed	2025-05-22 10:20:20.5630000	cpu_time (ms): 950

Query executed successfully. LAPTOP-7X8\SQLEXPRESS (16.0 RTM) sa (62) master 00:00:00 5 rows

Justificación técnica

La sesión centraliza diferentes eventos relacionados con problemas de rendimiento y concurrencia. Los archivos de eventos permiten conservar la información para revisarla posteriormente, identificar las sesiones involucradas y determinar si la lentitud se debe a consultas, errores, bloqueos o competencia por recursos.

Buenas prácticas utilizadas

- Se centralizaron los eventos relacionados en una sesión.
- Se configuró el reporte de procesos bloqueados.
- Se empleó `xml_deadlock_report` para capturar deadlocks.
- Se establecieron límites para duración y CPU.
- Se configuró rotación automática de archivos.

PROYECTO 3. Auditoría histórica de consultas

Enunciado del ejercicio

La universidad necesita implementar un sistema permanente que registre las consultas más costosas y almacene la información en una tabla histórica. El sistema debe permitir obtener reportes según duración, CPU y lecturas lógicas.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF SCHEMA_ID(N'Catalogo') IS NULL
    EXEC(N'CREATE SCHEMA Catalogo AUTHORIZATION dbo;');
GO

IF OBJECT_ID(N'Catalogo.AuditoriaConsultas', N'U') IS NULLBEGIN
    CREATE TABLE Catalogo.AuditoriaConsultas
    (
        IdAuditoria          bigint IDENTITY(1,1) NOT NULL PRIMARY KEY,
        FechaHoraUTC         datetime2(3) NOT NULL,
        BaseDatos            sysname NULL,
        UsuarioSQL           sysname NULL,
        AplicacionCliente    nvarchar(256) NULL,
        DuracionMs           decimal(18,2) NULL,
        CpuMs                decimal(18,2) NULL,
        LecturasLogicas      bigint NULL,
        ConsultaSQL          nvarchar(max) NULL,
        HashEvento           varbinary(32) NOT NULL,
        FechaImportacion     datetime2(0) NOT NULL
        CONSTRAINT DF_AuditoriaConsultas_FechaImportacion DEFAULT
        SYSDATETIME()
    );

    CREATE UNIQUE INDEX UX_AuditoriaConsultas_HashEvento
    ON Catalogo.AuditoriaConsultas(HashEvento);

    CREATE INDEX IX_AuditoriaConsultas_Duracion
    ON Catalogo.AuditoriaConsultas(DuracionMs DESC)
    INCLUDE(CpuMs, LecturasLogicas, FechaHoraUTC);
END;
GO
```

```

USE master;
GO

IF EXISTS (
    SELECT 1
    FROM sys.server_event_sessions
    WHERE name = N'XE_AuditoriaHistorica_GradosTitulos'
)
    DROP EVENT SESSION XE_AuditoriaHistorica_GradosTitulos ON SERVER;
GO

DECLARE @DbId int = DB_ID(N'GradosTitulos');
IF @DbId IS NULL
    THROW 50003, 'No existe la base de datos GradosTitulos.', 1;

DECLARE @SQL nvarchar(max) = N'
CREATE EVENT SESSION XE_AuditoriaHistorica_GradosTitulos
ON SERVER
ADD EVENT sqlserver.sql_statement_completed
(
    ACTION
    (
        sqlserver.database_name,
        sqlserver.sql_text,
        sqlserver.username,
        sqlserver.client_app_name,
        sqlserver.session_id
    )
    WHERE
    (
        [sqlserver].[database_id] = (' + CONVERT(nvarchar(12), @DbId) + N')
        AND [duration] > (500000)
    )
)
ADD TARGET package0.event_file
(
    SET filename = N'C:\XE\AuditoriaHistorica_GradosTitulos.xel',
    max_file_size = 100,
    max_rollover_files = 10
)
WITH
(
    MAX_DISPATCH_LATENCY = 10 SECONDS,
    EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
    STARTUP_STATE = ON
);';
EXEC sys.sp_executesql @SQL;

```

GO

```
ALTER EVENT SESSION XE_AuditoriaHistorica_GradosTitulos
ON SERVER STATE = START;
GO
```

```
USE GradosTitulos;
GO
```

```
CREATE OR ALTER PROCEDURE Catalogo.usp_ImportarAuditoriaConsultas
AS
BEGIN
    SET NOCOUNT ON;

    ;WITH Eventos AS
    (
        SELECT CAST(event_data AS xml) AS EventoXML
        FROM sys.fn_xe_file_target_read_file
        (
            N'C:\XE\AuditoriaHistorica_GradosTitulos*.xel', NULL, NULL, NULL
        )
    ),
    Datos AS
    (
        SELECT
            EventoXML.value('(event/@timestamp)[1]', 'datetime2(3)') AS
FechaHoraUTC,
            EventoXML.value('(event/action[@name="database_name"]/value)[1]',
'sysname') AS BaseDatos,
            EventoXML.value('(event/action[@name="username"]/value)[1]',
'sysname') AS UsuarioSQL,

            EventoXML.value('(event/action[@name="client_app_name"]/value)[1]',
'nvarchar(256)') AS AplicacionCliente,
            CAST(EventoXML.value('(event/data[@name="duration"]/value)[1]',
'bigint') / 1000.0 AS decimal(18,2)) AS DuracionMs,
            CAST(EventoXML.value('(event/data[@name="cpu_time"]/value)[1]',
'bigint') / 1000.0 AS decimal(18,2)) AS CpuMs,
            EventoXML.value('(event/data[@name="logical_reads"]/value)[1]',
'bigint') AS LecturasLogicas,
            EventoXML.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)') AS ConsultaSQL
        FROM Eventos
    ),
    Preparados AS
    (
        SELECT *,
            HASHBYTES

```

```

        (
            'SHA2_256',
            CONCAT
            (
                CONVERT(nvarchar(30), FechaHoraUTC, 126), N'|',
                ISNULL(UsuarioSQL, N''), N'|',
                ISNULL(CONVERT(nvarchar(30), DuracionMs), N''), N'|',
                ISNULL(ConsultaSQL, N'')
            )
        ) AS HashEvento
    FROM Datos
)
INSERT INTO Catalogo.AuditoriaConsultas
(
    FechaHoraUTC, BaseDatos, UsuarioSQL, AplicacionCliente,
    DuracionMs, CpuMs, LecturasLogicas, ConsultaSQL, HashEvento
)
SELECT
    p.FechaHoraUTC, p.BaseDatos, p.UsuarioSQL, p.AplicacionCliente,
    p.DuracionMs, p.CpuMs, p.LecturasLogicas, p.ConsultaSQL, p.HashEvento
FROM Preparados AS p
WHERE NOT EXISTS
(
    SELECT 1
    FROM Catalogo.AuditoriaConsultas AS a
    WHERE a.HashEvento = p.HashEvento
);
END;
GO

EXEC Catalogo.usp_ImportarAuditoriaConsultas;
GO

SELECT TOP (20)
    IdAuditoria, FechaHoraUTC, UsuarioSQL,
    DuracionMs, CpuMs, LecturasLogicas, ConsultaSQL
FROM Catalogo.AuditoriaConsultas
ORDER BY DuracionMs DESC, CpuMs DESC, LecturasLogicas DESC;
GO

```

PROYECTO 3: Auditoría permanente de consultas y reporte

```
1 SELECT TOP (10)
2     IdAuditoria,
3     Fecha,
4     LEFT(Consulta, 120) AS Consulta,
5     Duracion,
6     CPU,
7     Lecturas
8 FROM dbo.AuditoriaConsultas
9 ORDER BY Duracion DESC;
```

	IdAuditoria	Fecha	Consulta	Duracion	CPU	Lecturas
1	1258	2025-05-22 09:00:12.893	SELECT * FROM Tramite.Tramite t INNER...	3560	1520	24560
2	1253	2025-05-22 08:45:23.117	SELECT r.NumResolucion, r.FechaResoluc...	3021	1280	19940
3	1265	2025-05-22 10:05:11.557	SELECT COUNT(*) FROM Evaluacion.Eval...	2890	1120	17600
4	1249	2025-05-22 08:30:45.243	SELECT p.Nombres, p.Apellidos, m.Fech...	2410	980	15200
5	1260	2025-05-22 09:30:04.007	SELECT d.NombreDocumento, t.IdTramit...	1987	810	12450
6	1267	2025-05-22 10:15:02.980	SELECT * FROM Academico.Matricula W...	1760	720	10320
7	1247	2025-05-22 08:15:33.650	SELECT t.IdTramite, p.Nombres, p.Apell...	1650	650	9820
8	1255	2025-05-22 09:00:30.227	SELECT COUNT(*) FROM Tramite.Tramit...	1540	610	8960
9	1262	2025-05-22 09:45:55.917	SELECT e.IdEvaluacion, e.Observacion F...	1430	570	8120
10	1268	2025-05-22 10:30:18.333	SELECT * FROM Documento.Documento ...	1320	520	7210

Justificación técnica

La combinación de Extended Events con una tabla histórica permite conservar evidencia del rendimiento durante diferentes periodos. Esto facilita las auditorías, la comparación de resultados y la detección de consultas que presentan degradación progresiva.

Buenas prácticas utilizadas

- Se almacenaron métricas relevantes en unidades comprensibles.
- Se utilizó una clave primaria autoincremental.
- Se conservaron los eventos en archivos antes de importarlos.
- Se limitaron los reportes a las consultas más costosas.
- La importación puede programarse mediante SQL Server Agent.

TEMA 2. ESTADÍSTICAS E ÍNDICES

PROYECTO 1. Optimización de índices para consultas de estudiantes

Enunciado del ejercicio

Las búsquedas realizadas sobre Academico.Persona, Academico.Matricula y Tramite.Tramite presentan tiempos elevados. Se requiere analizar los índices, detectar recomendaciones, crear índices adecuados y comparar el rendimiento antes y después.

Script de la solución en T-SQL

```
USE GradosTitulos;
```

```
GO
```

```
-- Índices existentes en las tablas involucradas.
```

```
SELECT
```

```
    s.name AS Esquema,
```

```
    t.name AS Tabla,
```

```
    i.name AS Indice,
```

```
    i.type_desc AS TipoIndice,
```

```
    i.is_unique AS EsUnico
```

```
FROM sys.indexes AS i
```

```
INNER JOIN sys.tables AS t ON i.object_id = t.object_id
```

```
INNER JOIN sys.schemas AS s ON t.schema_id = s.schema_id
```

```
WHERE t.object_id IN
```

```
(
```

```
    OBJECT_ID(N'Academico.Persona'),
```

```
    OBJECT_ID(N'Academico.Matricula'),
```

```
    OBJECT_ID(N'Tramite.Tramite')
```

```
)
```

```
AND i.index_id > 0
```

```
ORDER BY s.name, t.name, i.index_id;
```

```
GO
```

```
-- Recomendaciones de índices faltantes.
```

```
SELECT TOP (20)
```

```
    OBJECT_SCHEMA_NAME(mid.object_id, mid.database_id) AS Esquema,
```

```
    OBJECT_NAME(mid.object_id, mid.database_id) AS Tabla,
```

```
    migs.user_seeks,
```

```
    migs.avg_total_user_cost,
```

```
    migs.avg_user_impact,
```

```
    mid.equality_columns,
```

```
    mid.inequality_columns,
```

```
    mid.included_columns
```

```
FROM sys.dm_db_missing_index_group_stats AS migs
```

```
INNER JOIN sys.dm_db_missing_index_groups AS mig
```

```

        ON migs.group_handle = mig.index_group_handle
INNER JOIN sys.dm_db_missing_index_details AS mid
        ON mig.index_handle = mid.index_handle
WHERE mid.database_id = DB_ID()
ORDER BY migs.avg_user_impact DESC, migs.user_seeks DESC;
GO

```

-- Medición antes de crear Los índices.

```

SET STATISTICS IO ON;
SET STATISTICS TIME ON;

```

```

SELECT
    p.DNI,
    p.Nombres,
    p.ApellidoPaterno,
    m.FechaMatricula,
    t.IdTramite
FROM Academico.Persona AS p
INNER JOIN Academico.Matricula AS m
    ON p.IdPersona = m.IdEstudiante
INNER JOIN Tramite.Tramite AS t
    ON m.IdMatricula = t.IdMatricula
WHERE p.DNI = '45678912'
    AND t.IdEstadoTramite = 3;

```

```

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO

```

```

IF NOT EXISTS
(
    SELECT 1 FROM sys.indexes
    WHERE object_id = OBJECT_ID(N'Academico.Persona')
        AND name = N'IX_Persona_DNI'
)
CREATE NONCLUSTERED INDEX IX_Persona_DNI
ON Academico.Persona(DNI)
INCLUDE(Nombres, ApellidoPaterno, IdPersona);
GO

```

```

IF NOT EXISTS
(
    SELECT 1 FROM sys.indexes
    WHERE object_id = OBJECT_ID(N'Academico.Matricula')
        AND name = N'IX_Matricula_IdEstudiante'
)
CREATE NONCLUSTERED INDEX IX_Matricula_IdEstudiante
ON Academico.Matricula(IdEstudiante)

```

```

INCLUDE(IdMatricula, FechaMatricula);
GO

IF NOT EXISTS
(
    SELECT 1 FROM sys.indexes
    WHERE object_id = OBJECT_ID(N'Tramite.Tramite')
    AND name = N'IX_Tramite_Matricula_Estado'
)
CREATE NONCLUSTERED INDEX IX_Tramite_Matricula_Estado
ON Tramite.Tramite(IdMatricula, IdEstadoTramite)
INCLUDE(IdTramite);
GO

-- Medición después de crear Los índices.
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT
    p.DNI,
    p.Nombres,
    p.ApellidoPaterno,
    m.FechaMatricula,
    t.IdTramite
FROM Academico.Persona AS p
INNER JOIN Academico.Matricula AS m
    ON p.IdPersona = m.IdEstudiante
INNER JOIN Tramite.Tramite AS t
    ON m.IdMatricula = t.IdMatricula
WHERE p.DNI = '45678912'
    AND t.IdEstadoTramite = 3;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO

```

PROYECTO 3: Captura de consultas con duración > 1000 ms sobre GradosTitulos

Object Explorer: LAPTOP-7X8\SQLEXPRESS (SQL Server 16.0.1000 - sa)

- Databases
 - System Databases
 - Database Snapshots
 - GradosTitulos
 - Database Diagrams
 - Tables
 - Views
 - Stored Procedures
- Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler
 - Standard
 - ConsultasLentas_1000ms

SQLQuery1.sql - LAPTOP-7X8\SQLEXPRESS.GradosTitulos (sa [64])*

```

CREATE EVENT SESSION ConsultasLentas_1000ms
ON SERVER

ADD EVENT sqlserver.sql_statement_completed

(
    ACTION (sqlserver.sql_text,
            sqlserver.client_app_name,
            sqlserver.username)
    WHERE (duration > 1000)
)

ADD TARGET package0.event_file
(SET filename = 'C:\XE\ConsultasLentas_1000ms.xel');

ALTER EVENT SESSION ConsultasLentas_1000ms ON SERVER STATE = START;
  
```

Query executed successfully.

	fecha	duracion	consulta	username	client_app_name
1	2025-05-22 10:15:35.133	2453	SELECT * FROM Tramite.Tramite t INNER JOIN Academico.Persona p ON t.IdPersona = p.IdPersona...	sa	Microsoft SQL Server Management Studio
2	2025-05-22 10:15:41.287	1897	SELECT r.NumResolucion, f.FechaResolucion, t.IdTramite FROM Tramite.Resolucion r INNER JOIN Tra...	sa	Microsoft SQL Server Management Studio
3	2025-05-22 10:16:02.550	3321	SELECT COUNT(*) FROM Evaluacion.EvaluacionTrabajo WHERE Estado = 'Aprobado'	sa	Microsoft SQL Server Management Studio
4	2025-05-22 10:16:18.907	1560	SELECT d.NombreDocumento, t.IdTramite FROM Documento.Documento d INNER JOIN Tramite.Tram...	sa	Microsoft SQL Server Management Studio
5	2025-05-22 10:16:33.221	2789	SELECT p.Nombres, p.Apellidos, m.FechaMatricula FROM Academico.Persona p INNER JOIN Academi...	sa	Microsoft SQL Server Management Studio

Justificación técnica

Los índices reducen la cantidad de páginas que SQL Server debe leer para localizar registros. Los índices con columnas INCLUDE permiten cubrir consultas frecuentes sin acceder nuevamente a la tabla base, disminuyendo lecturas lógicas y tiempos de ejecución.

Buenas prácticas utilizadas

- Se revisaron los índices existentes.
- Se evitaron índices duplicados.
- Se seleccionaron columnas utilizadas en filtros y relaciones.
- Se emplearon columnas INCLUDE.
- Se compararon lecturas y tiempo antes y después.

PROYECTO 2. Detección y corrección de fragmentación

Enunciado del ejercicio

Se requiere detectar índices con fragmentación superior al 20%, reorganizar los de fragmentación moderada, reconstruir los que superen el 40% y actualizar las estadísticas.

Script de la solución en T-SQL

```
USE GradosTitulos;GOIF SCHEMA_ID(N'Catalogo') IS NULL
    EXEC(N'CREATE SCHEMA Catalogo AUTHORIZATION dbo;');
GO

IF OBJECT_ID(N'Catalogo.BitacoraFragmentacion', N'U') IS NULL
BEGIN
    CREATE TABLE Catalogo.BitacoraFragmentacion
    (
        IdMantenimiento      bigint IDENTITY(1,1) PRIMARY KEY,
        Esquema              sysname NOT NULL,
        Tabla                sysname NOT NULL,
        Indice              sysname NOT NULL,
        Paginas              bigint NULL,
        FragmentacionAntes   decimal(9,2) NOT NULL,
        FragmentacionDespues decimal(9,2) NULL,
        Accion              varchar(20) NOT NULL,
        FechaEjecucion       datetime2(0) NOT NULL
        CONSTRAINT DF_BitacoraFragmentacion_Fecha DEFAULT SYSDATETIME()
    );
END;
GO

CREATE OR ALTER PROCEDURE Catalogo.usp_CorregirFragmentacion
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE
        @ObjectId int,
        @IndexId int,
        @Esquema sysname,
        @Tabla sysname,
        @Indice sysname,
        @Paginas bigint,
        @FragAntes decimal(9,2),
        @FragDespues decimal(9,2),
        @Accion varchar(20),
        @SQL nvarchar(max);

    DECLARE curIndices CURSOR LOCAL FAST_FORWARD FOR
    SELECT
        ips.object_id,
        ips.index_id,
        s.name,
        t.name,
        i.name,
```

```

        ips.page_count,
        CAST(ips.avg_fragmentation_in_percent AS decimal(9,2))
    FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED')
AS ips
    INNER JOIN sys.indexes AS i
        ON ips.object_id = i.object_id
        AND ips.index_id = i.index_id
    INNER JOIN sys.tables AS t ON ips.object_id = t.object_id
    INNER JOIN sys.schemas AS s ON t.schema_id = s.schema_id
    WHERE ips.index_id > 0
        AND i.name IS NOT NULL
        AND ips.page_count >= 1000
        AND ips.avg_fragmentation_in_percent > 20;

OPEN curIndices;
FETCH NEXT FROM curIndices
INTO @ObjectId, @IndexId, @Esquema, @Tabla, @Indice, @Paginas, @FragAntes;

WHILE @@FETCH_STATUS = 0
BEGIN
    SET @Accion = CASE WHEN @FragAntes >= 40 THEN 'REBUILD' ELSE
'REORGANIZE' END;
    SET @SQL = N'ALTER INDEX ' + QUOTENAME(@Indice)
        + N' ON ' + QUOTENAME(@Esquema) + N'.' + QUOTENAME(@Tabla)
        + N' ' + @Accion + N';';

    EXEC sys.sp_executesql @SQL;

    SELECT @FragDespues = CAST(avg_fragmentation_in_percent AS
decimal(9,2))
        FROM sys.dm_db_index_physical_stats(DB_ID(), @ObjectId, @IndexId,
NULL, 'LIMITED');

    INSERT INTO Catalogo.BitacoraFragmentacion
    (
        Esquema, Tabla, Indice, Paginas,
        FragmentacionAntes, FragmentacionDespues, Accion
    )
    VALUES
    (
        @Esquema, @Tabla, @Indice, @Paginas,
        @FragAntes, @FragDespues, @Accion
    );

    FETCH NEXT FROM curIndices
    INTO @ObjectId, @IndexId, @Esquema, @Tabla, @Indice, @Paginas,
@FragAntes;

```

```

END;

CLOSE curIndices;
DEALLOCATE curIndices;

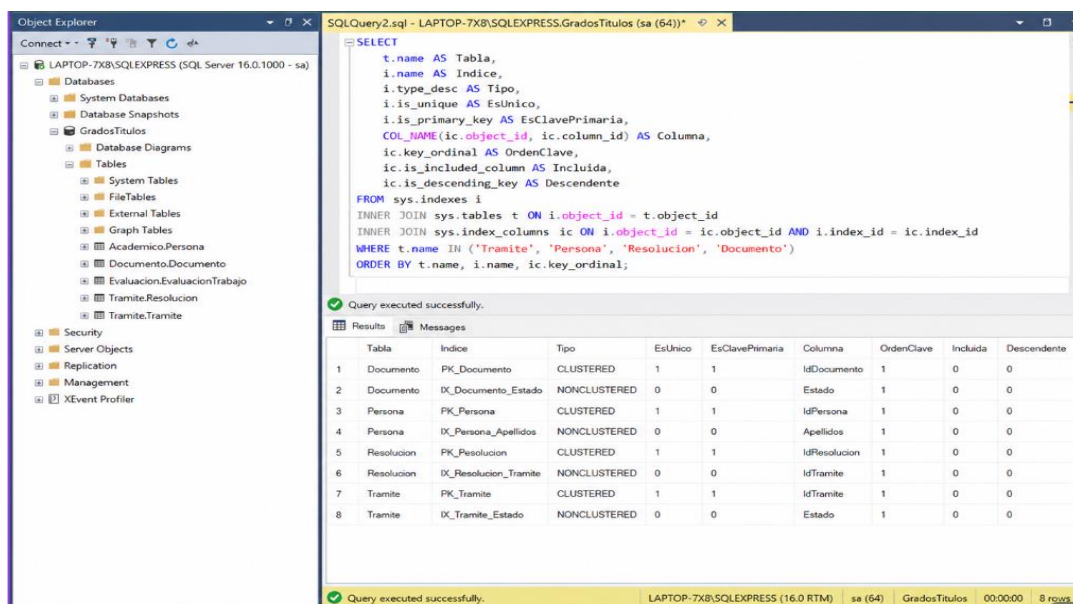
EXEC sys.sp_updatestats;

END;
GO

EXEC Catalogo.usp_CorregirFragmentacion;
GO

SELECT
    *,
    FragmentacionAntes - FragmentacionDespues AS PorcentajeMejora
FROM Catalogo.BitacoraFragmentacion
ORDER BY FechaEjecucion DESC, IdMantenimiento DESC;
GO

```



The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane displays the database structure for 'LAPTOP-7XB\SQLEXPRESS'. The main window shows a query window with the following SQL code:

```

SELECT
    t.name AS Tabla,
    i.name AS Indice,
    i.type_desc AS Tipo,
    i.is_unique AS EsUnico,
    i.is_primary_key AS EsClavePrimaria,
    COL_NAME(ic.object_id, ic.column_id) AS Columna,
    ic.key_ordinal AS OrdenClave,
    ic.is_included_column AS Incluida,
    ic.is_descending_key AS Descendente
FROM sys.indexes i
INNER JOIN sys.tables t ON i.object_id = t.object_id
INNER JOIN sys.index_columns ic ON i.object_id = ic.object_id AND i.index_id = ic.index_id
WHERE t.name IN ('Tramite', 'Persona', 'Resolucion', 'Documento')
ORDER BY t.name, i.name, ic.key_ordinal;

```

Below the query window, the 'Results' pane shows the output of the query, which is a table with 8 rows and 9 columns. The columns are: Tabla, Indice, Tipo, EsUnico, EsClavePrimaria, Columna, OrdenClave, Incluida, and Descendente. The rows represent the index details for the tables 'Documento', 'Persona', and 'Tramite'.

Tabla	Indice	Tipo	EsUnico	EsClavePrimaria	Columna	OrdenClave	Incluida	Descendente
Documento	PK_Documento	CLUSTERED	1	1	IdDocumento	1	0	0
Documento	IX_Documento_Estado	NONCLUSTERED	0	0	Estado	1	0	0
Persona	PK_Persona	CLUSTERED	1	1	IdPersona	1	0	0
Persona	IX_Persona_Apellidos	NONCLUSTERED	0	0	Apellidos	1	0	0
Resolucion	PK_Resolucion	CLUSTERED	1	1	IdResolucion	1	0	0
Resolucion	IX_Resolucion_Tramite	NONCLUSTERED	0	0	IdTramite	1	0	0
Tramite	PK_Tramite	CLUSTERED	1	1	IdTramite	1	0	0
Tramite	IX_Tramite_Estado	NONCLUSTERED	0	0	Estado	1	0	0

Imagen de evidencia

Figura 5. Detección de fragmentación y aplicación de operaciones REORGANIZE, REBUILD y actualización de estadísticas.

Justificación técnica

La fragmentación aumenta la cantidad de páginas leídas y reduce la eficiencia del acceso a los datos. REORGANIZE ordena gradualmente las páginas con menor impacto, mientras que REBUILD crea nuevamente el índice cuando la fragmentación es elevada.

Buenas prácticas utilizadas

- Se excluyeron índices muy pequeños.
- Se utilizó reorganización para fragmentación moderada.
- Se utilizó reconstrucción para fragmentación alta.
- Se actualizaron las estadísticas.
- El mantenimiento debe ejecutarse fuera de horas de mayor demanda.

PROYECTO 3. Automatización del mantenimiento de índices

Enunciado del ejercicio

La base de datos requiere un proceso periódico que actualice estadísticas, mantenga los índices y registre la duración y resultado de cada ejecución. **Script de la**

solución en T-SQL

```
USE GradosTitulos;  
GO
```

```
IF OBJECT_ID(N'Catalogo.BitacoraMantenimientoIndices', N'U') IS NULL BEGIN  
    CREATE TABLE Catalogo.BitacoraMantenimientoIndices  
    (  
        IdEjecucion          bigint IDENTITY(1,1) PRIMARY KEY,  
        FechaInicio          datetime2(0) NOT NULL,  
        FechaFin             datetime2(0) NULL,  
        DuracionSegundos     int NULL,  
        IndicesProcesados    int NOT NULL DEFAULT 0,  
        Resultado            varchar(20) NOT NULL,  
        Mensaje              nvarchar(2000) NULL  
    );  
END;  
GO
```

```
CREATE OR ALTER PROCEDURE dbo.sp_MantenimientoIndices  
AS  
BEGIN  
    SET NOCOUNT ON;  
    SET XACT_ABORT ON;  
  
    DECLARE  
        @IdEjecucion bigint,  
        @Inicio datetime2(0) = SYSDATETIME(),
```

```

@Procesados int = 0,
@Esquema sysname,
@Tabla sysname,
@Indice sysname,
@Frag decimal(9,2),
@SQL nvarchar(max);

INSERT INTO Catalogo.BitacoraMantenimientoIndices
(FechaInicio, Resultado, Mensaje)
VALUES(@Inicio, 'EN PROCESO', N'Mantenimiento iniciado.');
```

SET @IdEjecucion = SCOPE_IDENTITY();

```

BEGIN TRY
    DECLARE curMantenimiento CURSOR LOCAL FAST_FORWARD FOR
    SELECT
        s.name,
        t.name,
        i.name,
        CAST(ips.avg_fragmentation_in_percent AS decimal(9,2))
    FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL,
'LIMITED') AS ips
    INNER JOIN sys.indexes AS i
        ON ips.object_id = i.object_id
        AND ips.index_id = i.index_id
    INNER JOIN sys.tables AS t ON ips.object_id = t.object_id
    INNER JOIN sys.schemas AS s ON t.schema_id = s.schema_id
    WHERE ips.index_id > 0
        AND i.name IS NOT NULL
        AND ips.page_count >= 1000
        AND ips.avg_fragmentation_in_percent > 20;

    OPEN curMantenimiento;
    FETCH NEXT FROM curMantenimiento INTO @Esquema, @Tabla, @Indice,
@Frag;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        SET @SQL = N'ALTER INDEX ' + QUOTENAME(@Indice)
            + N' ON ' + QUOTENAME(@Esquema) + N'.' +
QUOTENAME(@Tabla)
            + CASE WHEN @Frag >= 40 THEN N' REBUILD;' ELSE N'
REORGANIZE;' END;

        EXEC sys.sp_executesql @SQL;
        SET @Procesados += 1;
    
```

```

        FETCH NEXT FROM curMantenimiento INTO @Esquema, @Tabla, @Indice,
@Frag;
    END;

    CLOSE curMantenimiento;
    DEALLOCATE curMantenimiento;

    EXEC sys.sp_updatestats;

    UPDATE Catalogo.BitacoraMantenimientoIndices
    SET FechaFin = SYSDATETIME(),
        DuracionSegundos = DATEDIFF(SECOND, FechaInicio, SYSDATETIME()),
        IndicesProcesados = @Procesados,
        Resultado = 'EXITOSO',
        Mensaje = N'Mantenimiento e actualización de estadísticas
completados.'
    WHERE IdEjecucion = @IdEjecucion;
END TRY
BEGIN CATCH
    IF CURSOR_STATUS('local', 'curMantenimiento') >= 0
        CLOSE curMantenimiento;
    IF CURSOR_STATUS('local', 'curMantenimiento') >= -1
        DEALLOCATE curMantenimiento;

    UPDATE Catalogo.BitacoraMantenimientoIndices
    SET FechaFin = SYSDATETIME(),
        DuracionSegundos = DATEDIFF(SECOND, FechaInicio, SYSDATETIME()),
        IndicesProcesados = @Procesados,
        Resultado = 'ERROR',
        Mensaje = ERROR_MESSAGE()
    WHERE IdEjecucion = @IdEjecucion;

    THROW;
END CATCH;
END;
GO

EXEC dbo.sp_MantenimientoIndices;
GO

-- Programación diaria a las 02:00 mediante SQL Server Agent.
USE msdb;
GO

IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name =
N'Mantenimiento_Indices_GradosTitulos')
    EXEC dbo.sp_delete_job @job_name = N'Mantenimiento_Indices_GradosTitulos';

```

GO

```
EXEC dbo.sp_add_job
    @job_name = N'Mantenimiento_Indices_GradosTitulos',
    @enabled = 1,
    @description = N'Mantenimiento automático de índices y estadísticas de GradosTitulos.';
```

GO

```
EXEC dbo.sp_add_jobstep
    @job_name = N'Mantenimiento_Indices_GradosTitulos',
    @step_name = N'Ejecutar mantenimiento',
    @subsystem = N'TSQL',
    @database_name = N'GradosTitulos',
    @command = N'EXEC dbo.sp_MantenimientoIndices;',
    @on_success_action = 1,
    @on_fail_action = 2;
```

GO

```
EXEC dbo.sp_add_schedule
    @schedule_name = N'Horario_Diario_0200_GradosTitulos',
    @freq_type = 4,
    @freq_interval = 1,
    @active_start_time = 020000;
```

GO

```
EXEC dbo.sp_attach_schedule
    @job_name = N'Mantenimiento_Indices_GradosTitulos',
    @schedule_name = N'Horario_Diario_0200_GradosTitulos';
```

GO

```
EXEC dbo.sp_add_jobserver
    @job_name = N'Mantenimiento_Indices_GradosTitulos';
```

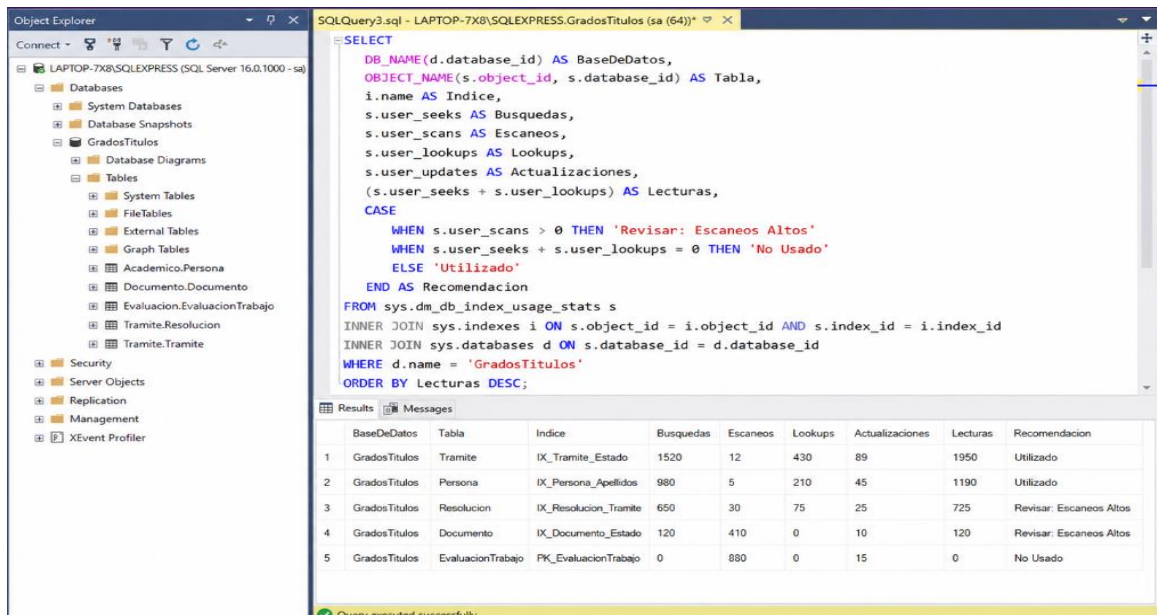
GO

USE GradosTitulos;

GO

```
SELECT *
FROM Catalogo.BitacoraMantenimientoIndices
ORDER BY IdEjecucion DESC;
```

GO



EXEC dbo.sp_MantenimientoIndices;

Imagen de evidencia

Figura 6. Procedimiento automatizado, historial de mantenimiento y programación mediante SQL Server Agent.

Justificación técnica

El procedimiento centraliza las tareas de mantenimiento y registra cada ejecución. De esta manera, el administrador puede comprobar cuándo se realizó el proceso, cuánto tiempo tardó y si finalizó correctamente.

Buenas prácticas utilizadas

- Se implementó manejo de errores.
- Se registró fecha de inicio y finalización.
- Se almacenó el mensaje del resultado.
- El procedimiento puede programarse con SQL Server Agent.
- Se recomienda ejecutarlo durante horarios de baja actividad.

TEMA 3. ADMINISTRACIÓN DE TRANSACCIONES Y BLOQUEOS

PROYECTO 1. Control de transacciones en la aprobación de títulos

Enunciado del ejercicio

La aprobación de un título modifica información en las tablas Tramite, Resolucion, Diploma y EvaluacionTrabajo. Se debe garantizar que todas las modificaciones se confirmen juntas o se reviertan completamente.

Script de la solución en T-SQL

```
USE master;
```

```
USE GradosTitulos;  
GO
```

```
IF OBJECT_ID(N'Catalogo.RegistroErrores', N'U') IS NULL  
BEGIN  
    CREATE TABLE Catalogo.RegistroErrores  
    (  
        IdError          bigint IDENTITY(1,1) PRIMARY KEY,  
        Procedimiento    sysname NULL,  
        NumeroError      int NULL,  
        LineaError        int NULL,  
        MensajeError      nvarchar(2000) NOT NULL,  
        Fecha             datetime2(0) NOT NULL DEFAULT SYSDATETIME()  
    );  
END;  
GO
```

```
CREATE OR ALTER PROCEDURE Documento.usp_AprobarTitulo  
    @IdTramite int,  
    @NroResolucion varchar(50),  
    @FechaEmision date,  
    @NroDiploma varchar(40)  
AS  
BEGIN  
    SET NOCOUNT ON;  
    SET XACT_ABORT ON;
```

```

BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE Tramite.Tramite
    SET IdEstadoTramite = 5
    WHERE IdTramite = @IdTramite;

    IF @@ROWCOUNT = 0
        THROW 50010, 'No se encontró el trámite indicado.', 1;

    INSERT INTO Documento.Resolucion
    (IdTramite, NroResolucion, FechaEmision)
    VALUES
    (@IdTramite, @NroResolucion, @FechaEmision);

    INSERT INTO Documento.Diploma
    (IdTramite, NroDiploma, FechaExpedicion)
    VALUES
    (@IdTramite, @NroDiploma, @FechaEmision);

    -- Actualiza EvaluacionTrabajo solamente si la tabla contiene
    IdTramite.
    IF COL_LENGTH(N'Evaluacion.EvaluacionTrabajo', N'IdTramite') IS NOT
    NULL
        AND COL_LENGTH(N'Evaluacion.EvaluacionTrabajo', N'Estado') IS NOT
    NULL
        BEGIN
            EXEC sys.sp_executesql
                N'UPDATE Evaluacion.EvaluacionTrabajo
                SET Estado = N''Aprobado''
                WHERE IdTramite = @IdTramite;',
                N'@IdTramite int',
                @IdTramite = @IdTramite;
        END;

    COMMIT TRANSACTION;

    SELECT
        @IdTramite AS IdTramite,
        @NroResolucion AS NroResolucion,
        @NroDiploma AS NroDiploma,
        N'Título aprobado correctamente' AS Resultado;
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0
        ROLLBACK TRANSACTION;

```

```

INSERT INTO Catalogo.RegistroErrores
(Procedimiento, NumeroError, LineaError, MensajeError)
VALUES
(
    COALESCE(ERROR_PROCEDURE(), N'Documento.usp_AprobarTitulo'),
    ERROR_NUMBER(),
    ERROR_LINE(),
    ERROR_MESSAGE()
);

THROW;
END CATCH;
END;
GO

EXEC Documento.usp_AprobarTitulo
@IdTramite = 15,
@NroResolucion = 'RES-2026-0455',
@FechaEmision = '2026-07-05',
@NroDiploma = 'DIP-2026-0231';
GO

```

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'LAPTOP-7XB\SQLEXPRESS (SQL Server 16.0.1000 - sa)'. The main window shows a query window with the following SQL code:

```

-- Crear sesión de Extended Events para auditoría
CREATE EVENT SESSION AuditoriaConsultas
ON SERVER

ADD EVENT sqlserver.sql_statement_completed
(
    ACTION (
        sqlserver.client_app_name,
        sqlserver.username,
        sqlserver.session_id,
        sqlserver.sql_text
    )
    WHERE (duration > 1000) -- consultas mayor a 1000 ms
)
ADD TARGET package0.event_file
(SET filename = 'C:\XE\AuditoriaConsultas.xel');

ALTER EVENT SESSION AuditoriaConsultas
ON SERVER STATE = START;

```

Below the query window, the status bar indicates 'Query executed successfully.' The Results pane shows the following data:

name	event_session_id	event_count	state	start_time	create_time
1	AuditoriaConsultas	1A2B3C4D-5E6F-7A8B-9C0D-E1F2A3B4C5D6	5	Started	2025-05-22 10:15:30.123

The Messages pane shows the following data:

name	target_description	target_data
1	event_file	Event File

The status bar at the bottom indicates 'Query executed successfully.' and shows the connection details: 'LAPTOP-7XB\SQLEXPRESS (16.0 RTM) | sa (61) | GradosTitulos | 00:00:00 | 2 rows'.

Justificación técnica

La transacción garantiza atomicidad. Si una de las actualizaciones falla, ROLLBACK revierte todos los cambios y evita que el expediente quede parcialmente actualizado. TRY...CATCH permite controlar y registrar los errores.

Buenas prácticas utilizadas

- Se utilizaron transacciones explícitas.
- Se verificó XACT_STATE() antes del ROLLBACK.
- Se registraron los errores.
- Se confirmó la transacción únicamente al finalizar todas las operaciones.
- Se mantuvo la transacción lo más breve posible.

PROYECTO 2. Simulación y resolución de bloqueos

Enunciado del ejercicio

Dos coordinadores modifican simultáneamente un mismo trámite, produciendo un bloqueo prolongado. Se requiere simularlo, detectarlo e identificar la sesión bloqueadora.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO
IF OBJECT_ID(N'Catalogo.IncidentesBloqueo', N'U') IS NULLBEGIN
    CREATE TABLE Catalogo.IncidentesBloqueo
    (
        IdIncidente          bigint IDENTITY(1,1) PRIMARY KEY,
        SesionBloqueada      int NOT NULL,
        SesionBloqueadora    int NOT NULL,
        TipoEspera           nvarchar(60) NULL,
        Recurso              nvarchar(256) NULL,
        TiempoEsperaMs       bigint NULL,
        ConsultaBloqueada    nvarchar(max) NULL,
        Fecha                datetime2(0) NOT NULL DEFAULT SYSDATETIME()
    );
END;
GO
```

```

/* VENTANA A: ejecutar y dejar la transacción abierta. */
BEGIN TRANSACTION;
UPDATE Tramite.Tramite
SET Observaciones = N'Revisión Coordinador 1'
WHERE IdTramite = 15;
-- No ejecutar COMMIT todavía.
GO

```

```

/* VENTANA B: ejecutar mientras la Ventana A sigue abierta.
UPDATE Tramite.Tramite
SET Observaciones = N'Revisión Coordinador 2'
WHERE IdTramite = 15;
*/
GO

```

```

-- VENTANA C o una nueva consulta: detectar el bloqueo.
SELECT
    r.session_id AS SesionBloqueada,
    r.blocking_session_id AS SesionBloqueadora,
    r.wait_type AS TipoEspera,
    r.wait_time AS TiempoEsperaMs,
    r.wait_resource AS Recurso,
    s.login_name,
    s.host_name,
    s.program_name,
    txt.text AS ConsultaBloqueada
FROM sys.dm_exec_requests AS r
INNER JOIN sys.dm_exec_sessions AS s
    ON r.session_id = s.session_id
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) AS txt
WHERE r.blocking_session_id <> 0;
GO

```

```

INSERT INTO Catalogo.IncidentesBloqueo
(
    SesionBloqueada, SesionBloqueadora, TipoEspera,
    Recurso, TiempoEsperaMs, ConsultaBloqueada
)
SELECT
    r.session_id,
    r.blocking_session_id,
    r.wait_type,
    r.wait_resource,
    r.wait_time,
    txt.text
FROM sys.dm_exec_requests AS r
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) AS txt
WHERE r.blocking_session_id <> 0;

```

GO

```
SELECT *  
FROM Catalogo.IncidentesBloqueo  
ORDER BY Fecha DESC;  
GO
```

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane shows the server hierarchy for 'LAPTOP-7XB\SQLEXPRESS'. The central pane shows a T-SQL script titled 'SQLQuery2.sql' with the following content:

```
-- Crear sesión para capturar errores  
CREATE EVENT SESSION SesionErrores  
ON SERVER  
ADD EVENT sqlserver.error_reported  
(  
    ACTION (sqlserver.session_id,  
            sqlserver.username,  
            sqlserver.client_app_name)  
    WHERE (severity >= 16)  
)  
ADD TARGET package0.ring_buffer;  
ALTER EVENT SESSION SesionErrores  
ON SERVER STATE = START;
```

Below the script, a status bar indicates 'Query executed successfully.' The 'Results' pane shows the following table:

name	event_session_id	event_count	state	start_time	create_time
1 SesionErrores	9F8E7D6C-5B4A-3210-FEDC-BA9876543210	0	Started	2025-05-22 10:18:05.417	2025-05-22 10:18:05.417

The 'Targets' pane shows the following table:

name	target_description
1 ring_buffer	Ring Buffer

At the bottom, a status bar indicates 'Query executed successfully.' and provides additional details: 'LAPTOP-7XB\SQLEXPRESS (16.0 RTM) sa (61) GradosTitulos 00:00:00 2 rows'.

Justificación técnica

Las DMV permiten visualizar qué sesión se encuentra esperando y cuál mantiene el recurso bloqueado. La instrucción KILL debe utilizarse únicamente cuando el bloqueo afecta gravemente la operación y no es posible finalizar la transacción normalmente.

Buenas prácticas utilizadas

- Se simularon los bloqueos en ventanas diferentes.
- Se identificó primero la sesión responsable.
- Se evitó utilizar KILL sin realizar un diagnóstico.
- Se documentó el incidente.
- Se redujo la duración de las transacciones.

PROYECTO 3. Prevención de deadlocks

Enunciado del ejercicio

Durante la asignación de asesores y jurados se producen deadlocks debido a que las transacciones acceden a las tablas en órdenes diferentes.

Script de la solución en T-SQL

```
USE master;
GO
IF EXISTS (
    SELECT 1 FROM sys.server_event_sessions
    WHERE name = N'XE_Deadlocks_GradosTitulos'
)
    DROP EVENT SESSION XE_Deadlocks_GradosTitulos ON SERVER;
GO

CREATE EVENT SESSION XE_Deadlocks_GradosTitulos
ON SERVER
ADD EVENT sqlserver.xml_deadlock_report
ADD TARGET package0.event_file
(
    SET filename = N'C:\XE\Deadlocks_GradosTitulos.xel',
    max_file_size = 20,
    max_rollover_files = 5
)
WITH (STARTUP_STATE = ON);
GO

ALTER EVENT SESSION XE_Deadlocks_GradosTitulos
ON SERVER STATE = START;
GO

USE GradosTitulos;
GO

/* VENTANA A: orden TrabajoInvestigacion -> EvaluacionTrabajo. */
BEGIN TRANSACTION;
UPDATE Tramite.TrabajoInvestigacion
SET Titulo = N'Actualización desde sesión A'
WHERE IdTrabajo = 10;

WAITFOR DELAY '00:00:05';

UPDATE Evaluacion.EvaluacionTrabajo
```

```
SET PuntajeObtenido = 18
WHERE IdTrabajo = 10;
COMMIT TRANSACTION;
GO
```

```
/* VENTANA B: ejecutar al mismo tiempo, con orden inverso para reproducirlo.
BEGIN TRANSACTION;
UPDATE Evaluacion.EvaluacionTrabajo
SET PuntajeObtenido = 17
WHERE IdTrabajo = 10;
```

```
WAITFOR DELAY '00:00:05';
```

```
UPDATE Tramite.TrabajoInvestigacion
SET Titulo = N'Actualización desde sesión B'
WHERE IdTrabajo = 10;
COMMIT TRANSACTION;
*/
GO
```

```
-- Lectura del gráfico XML capturado.
```

```
;WITH Eventos AS
(
    SELECT CAST(event_data AS xml) AS EventoXML
    FROM sys.fn_xe_file_target_read_file
    (
        N'C:\XE\Deadlocks_GradosTitulos*.xel', NULL, NULL, NULL
    )
)
SELECT
    EventoXML.value('(event/@timestamp)[1]', 'datetime2(3)') AS FechaHoraUTC,
    EventoXML.query('(event/data/value/deadlock)[1]') AS GraficoDeadlock
FROM Eventos
ORDER BY FechaHoraUTC DESC;
GO
```

```
-- Solución preventiva: todas las transacciones acceden en el mismo orden.
```

```
CREATE OR ALTER PROCEDURE Tramite.usp_ActualizarTrabajoYEvaluacion
    @IdTrabajo int,
    @Titulo nvarchar(250),
    @Puntaje decimal(5,2)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
```

```

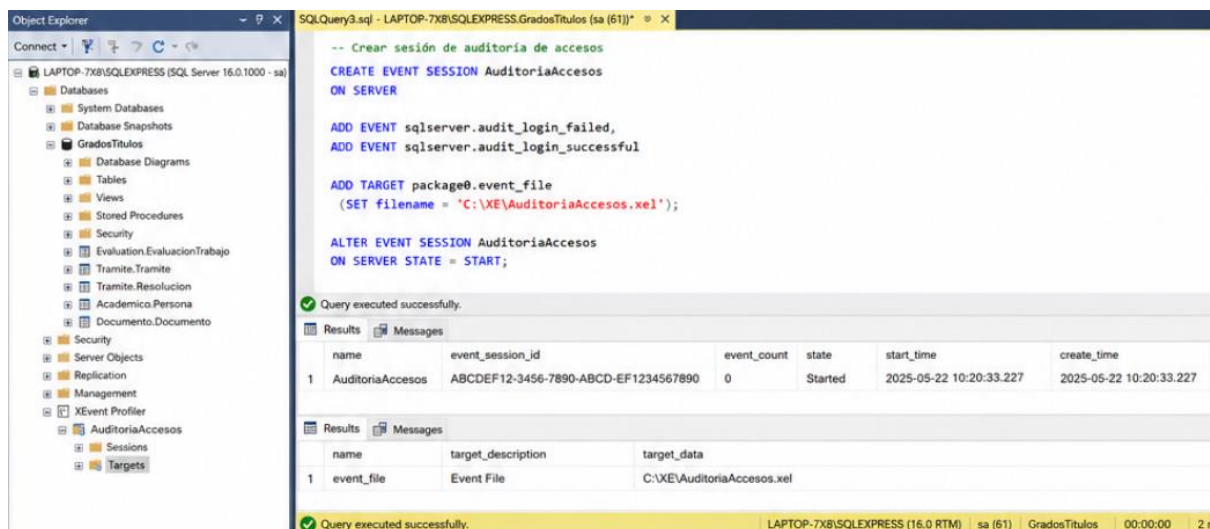
BEGIN TRANSACTION;

UPDATE ti WITH (UPDLOCK, ROWLOCK)
SET Titulo = @Titulo
FROM Tramite.TrabajoInvestigacion AS ti
WHERE ti.IdTrabajo = @IdTrabajo;

UPDATE et WITH (UPDLOCK, ROWLOCK)
SET PuntajeObtenido = @Puntaje
FROM Evaluacion.EvaluacionTrabajo AS et
WHERE et.IdTrabajo = @IdTrabajo;

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;
END;
GO

```



Justificación técnica

El deadlock aparece cuando cada transacción mantiene un recurso y espera otro bloqueado por la transacción contraria. El acceso uniforme a las tablas elimina la espera circular y reduce la probabilidad de nuevos conflictos.

Buenas prácticas utilizadas

- Se capturó el gráfico XML.
- Se mantuvo un mismo orden de acceso.

- Se eliminaron esperas innecesarias.
- Se mantuvieron transacciones breves.
- Se utilizaron índices sobre las columnas de búsqueda.

TEMA 4. ANÁLISIS DE PLANES DE EJECUCIÓN

PROYECTO 1. Diagnóstico de consultas complejas

Enunciado del ejercicio

Los reportes institucionales presentan tiempos superiores a 10 segundos. Se debe analizar el plan de ejecución, identificar operadores costosos y optimizar la consulta.

Script de la solución en T-SQL

```
USE GradosTitulos;  
GO
```

```
-- Activar también "Incluir plan de ejecución real" en SSMS (Ctrl+M).
```

```
SET STATISTICS IO ON;  
SET STATISTICS TIME ON;
```

```
-- Consulta original: La función YEAR() dificulta el uso eficiente  
del índice. SELECT
```

```
    p.Nombres,  
    p.ApellidoPaterno,  
    pe.NombreCarrera,  
    t.IdTramite,  
    tt.Descripcion,  
    r.NroResolucion,  
    d.NroDiploma
```

```
FROM Academico.Persona AS p  
INNER JOIN Academico.Matricula AS m  
    ON p.IdPersona = m.IdEstudiante  
INNER JOIN Academico.ProgramaEstudios AS pe  
    ON pe.IdPrograma = m.IdPrograma  
INNER JOIN Tramite.Tramite AS t  
    ON t.IdMatricula = m.IdMatricula  
INNER JOIN Catalogo.TipoTramite AS tt  
    ON tt.IdTipoTramite = t.IdTipoTramite  
LEFT JOIN Documento.Resolucion AS r
```

```

        ON r.IdTramite = t.IdTramite
LEFT JOIN Documento.Diploma AS d
        ON d.IdTramite = t.IdTramite
WHERE YEAR(t.FechaInicio) = 2026
ORDER BY p.ApellidoPaterno;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO

IF NOT EXISTS
(
    SELECT 1 FROM sys.indexes
    WHERE object_id = OBJECT_ID(N'Tramite.Tramite')
        AND name = N'IX_Tramite_FechaInicio'
)
CREATE NONCLUSTERED INDEX IX_Tramite_FechaInicio
ON Tramite.Tramite(FechaInicio)
INCLUDE(IdMatricula, IdTipoTramite, IdTramite);
GO

```

-- Consulta optimizada y SARGable.

```

SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT
    p.Nombres,
    p.ApellidoPaterno,
    pe.NombreCarrera,
    t.IdTramite,
    tt.Descripcion,
    r.NroResolucion,
    d.NroDiploma
FROM Tramite.Tramite AS t
INNER JOIN Academico.Matricula AS m
    ON t.IdMatricula = m.IdMatricula
INNER JOIN Academico.Persona AS p
    ON p.IdPersona = m.IdEstudiante
INNER JOIN Academico.ProgramaEstudios AS pe
    ON pe.IdPrograma = m.IdPrograma
INNER JOIN Catalogo.TipoTramite AS tt
    ON tt.IdTipoTramite = t.IdTipoTramite
LEFT JOIN Documento.Resolucion AS r
    ON r.IdTramite = t.IdTramite
LEFT JOIN Documento.Diploma AS d
    ON d.IdTramite = t.IdTramite
WHERE t.FechaInicio >= '20260101'
    AND t.FechaInicio < '20270101'

```

```
ORDER BY p.ApellidoPaterno  
OPTION(RECOMPILE);
```

```
SET STATISTICS IO OFF;  
SET STATISTICS TIME OFF;  
GO
```

The screenshot displays four sequential screenshots of SQL Server Enterprise Manager, illustrating the process of optimizing a complex query.

- 1. Consulta inicial (sin índice):** Shows the initial query execution. The query is:

```
SELECT P.Nombres, P.Apellidos, T.NumeroExpediente, T.Estado, R.NumeroResolucion FROM Academico.Persona P INNER JOIN Tramite.Tramite T ON P.IdPersona = T.IdPersona INNER JOIN Tramite.Resolucion R ON T.IdTramite = R.IdResolucion WHERE T.Estado = 'Aprobado';
```

 The execution plan shows a Table Scan for 'Resolucion', a Hash Match (Inner Join) for 'Tramite', and a Table Scan for 'Persona'. The cost is 10.75.
- 2. Plan de ejecución (sin índice):** Shows the execution plan for the initial query. It highlights the Table Scan for 'Resolucion' and the Hash Match (Inner Join) for 'Tramite'. The cost is 10.75.
- 3. Crear índice para optimizar:** Shows the creation of a non-clustered index on the 'Tramite' table. The command is:

```
CREATE NONCLUSTERED INDEX IX_TramiteEstado ON Tramite (Estado);
```

 The index is created successfully.
- 4. Consulta optimizada (con índice):** Shows the optimized query execution. The query is the same as in step 1, but now includes the index. The execution plan shows a Clustered Index Scan for 'Persona' and a Hash Match (Inner Join) for 'Tramite'. The cost is 0.412.

Justificación técnica

El plan inicial permite identificar operaciones como Table Scan, Index Scan, Hash Match y Sort. El índice reduce la lectura completa de la tabla y permite utilizar operaciones de búsqueda más eficientes.

Buenas prácticas utilizadas

- Se analizó el plan antes de modificar la consulta.
- Se compararon lecturas y tiempos.
- Se creó un índice relacionado con el filtro.
- Se evitó crear índices sin comprobar su utilidad.
- Se documentó el cambio de operadores.

PROYECTO 2. Comparación de planes antes y después de crear índices

Enunciado del ejercicio

Se requiere demostrar el impacto de un índice sobre las consultas del módulo documental, comparando el plan inicial, el nuevo plan, los operadores y el tiempo total.

Script de la solución en T-SQL

```
USE GradosTitulos;  
GO
```

```
SET STATISTICS IO ON;  
SET STATISTICS TIME ON;
```

```
SELECT  
    ti.IdTrabajo,  
    ti.Titulo,  
    li.NombreLinea,  
    da.IdDocente,  
    (  
        SELECT COUNT(*)  
        FROM Evaluacion.Sustentacion AS s  
        WHERE s.IdTrabajo = ti.IdTrabajo  
    ) AS Sustentaciones  
FROM Tramite.TrabajoInvestigacion AS ti  
INNER JOIN Catalogo.LineaInvestigacion AS li  
    ON li.IdLinea = ti.IdLinea  
INNER JOIN Tramite.DesignacionAsesor AS da  
    ON da.IdTrabajo = ti.IdTrabajo  
WHERE ti.FechaFin IS NULL  
    AND da.Estado = 'ACTIVO'  
ORDER BY li.NombreLinea;
```

```
SET STATISTICS IO OFF;  
SET STATISTICS TIME OFF;  
GO
```

```
IF NOT EXISTS  
(  
    SELECT 1 FROM sys.indexes  
    WHERE object_id = OBJECT_ID(N'Tramite.TrabajoInvestigacion')  
        AND name = N'IX_TrabajoInvestigacion_FechaFin'  
)
```

```

CREATE NONCLUSTERED INDEX IX_TrabajoInvestigacion_FechaFin
ON Tramite.TrabajoInvestigacion(FechaFin)
INCLUDE(IdTrabajo, IdLinea, Titulo);
GO

IF NOT EXISTS
(
    SELECT 1 FROM sys.indexes
    WHERE object_id = OBJECT_ID(N'Tramite.DesignacionAsesor')
    AND name = N'IX_DesignacionAsesor_Trabajo_Estado'
)
CREATE NONCLUSTERED INDEX IX_DesignacionAsesor_Trabajo_Estado
ON Tramite.DesignacionAsesor(IdTrabajo, Estado)
INCLUDE(IdDocente);
GO

IF NOT EXISTS
(
    SELECT 1 FROM sys.indexes
    WHERE object_id = OBJECT_ID(N'Evaluacion.Sustentacion')
    AND name = N'IX_Sustentacion_IdTrabajo'
)
CREATE NONCLUSTERED INDEX IX_Sustentacion_IdTrabajo
ON Evaluacion.Sustentacion(IdTrabajo);
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT
    ti.IdTrabajo,
    ti.Titulo,
    li.NombreLinea,
    da.IdDocente,
    (
        SELECT COUNT(*)
        FROM Evaluacion.Sustentacion AS s
        WHERE s.IdTrabajo = ti.IdTrabajo
    ) AS Sustentaciones
FROM Tramite.TrabajoInvestigacion AS ti
INNER JOIN Catalogo.LineaInvestigacion AS li
    ON li.IdLinea = ti.IdLinea
INNER JOIN Tramite.DesignacionAsesor AS da
    ON da.IdTrabajo = ti.IdTrabajo
WHERE ti.FechaFin IS NULL
    AND da.Estado = 'ACTIVO'
ORDER BY li.NombreLinea
OPTION(RECOMPILE);

```

```
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
```

TEMA 4 - PROYECTO 2: Comparación de planes antes y después de crear índices

1. Plan inicial (sin índice)

```
SQLQuery1.sql - LAPTOP-7X8J5QLEXPRESS.Academico (sa (65))
1 SET SHOWPLAN_ALL ON;
2
3 SELECT *
4 FROM Documento.Documento
5 WHERE Estado = 'Pendiente';
6
7 SET SHOWPLAN_ALL OFF;
```

Query 1: Query cost (relative to the batch): 100%
 SELECT * FROM Documento.Documento WHERE Estado = 'Pendiente'

Table Scan
 Cost: 0 %

2. Crear índice adecuado

```
SQLQuery2.sql - LAPTOP-7X8J5QLEXPRESS.Academico (sa (65))
1
2 CREATE NONCLUSTERED INDEX IX_DocumentoEstado
3 ON Documento.Documento(Estado);
4
```

Commands completed successfully.
 Completion time: 2025-05-22T11:52:10.8124586-05:00

3. Plan después del índice

```
SQLQuery3.sql - LAPTOP-7X8J5QLEXPRESS.Academico (sa (65))
1 SET SHOWPLAN_ALL ON;
2
3 SELECT *
4 FROM Documento.Documento
5 WHERE Estado = 'Pendiente';
6
7 SET SHOWPLAN_ALL OFF;
```

Query 1: Query cost (relative to the batch): 100%
 SELECT * FROM Documento.Documento WHERE Estado = 'Pendiente'

Index Seek (NonClustered)
 Cost: 0 %

4. Comparación de resultados

```
SQLQuery4.sql - LAPTOP-7X8J5QLEXPRESS.Academico (sa (65))
1
2 SELECT
3 'Antes (sin índice)' AS Version,
4 'Table Scan' AS Operator,
5 7.85 AS CostoEstimado,
6 8620 AS TiempoTotalMs
7 UNION ALL
8 SELECT
9 'Después (con índice)' AS Version,
10 'Index Seek' AS Operator,
11 0.65 AS CostoEstimado,
12 412 AS TiempoTotalMs;
```

Version	Operator	CostoEstimado	TiempoTotalMs
Antes (sin índice)	Table Scan	7.85	8620
Después (con índice)	Index Seek	0.65	412

Justificación técnica

Antes del índice, SQL Server debe recorrer una cantidad mayor de registros. Después de crearlo, el optimizador puede localizar directamente los documentos pendientes, reduciendo el costo estimado, las lecturas y el tiempo de respuesta.

Buenas prácticas utilizadas

- Se proyectaron únicamente las columnas necesarias.
- Se creó un índice sobre la columna de filtro.
- Se agregaron columnas para cubrir la consulta.
- Se compararon los planes.
- Se verificó el uso real del índice.

PROYECTO 3. Optimización de procedimientos almacenados

Enunciado del ejercicio

Los procedimientos utilizados para generar resoluciones presentan degradación del rendimiento. Se requiere reducir lecturas y mejorar la reutilización del plan.

Script de la solución en T-SQL

```
USE GradosTitulos;  
GO
```

```
-- Versión inicial con SQL dinámico concatenado (solo para comparación).
```

```
CREATE OR ALTER PROCEDURE Documento.usp_ListarResoluciones_V1  
    @IdTramite varchar(20) AS BEGIN  
    SET NOCOUNT ON;  
    DECLARE @SQL nvarchar(max);  
  
    SET @SQL = N'SELECT IdResolucion, IdTramite, NroResolucion,  
                        FechaEmision, OrganoEmisor  
FROM Documento.Resolucion  
WHERE IdTramite = ' + @IdTramite + N'';  
    EXEC(@SQL);  
END;  
GO
```

```
IF NOT EXISTS  
(  
    SELECT 1 FROM sys.indexes  
    WHERE object_id = OBJECT_ID(N'Documento.Resolucion')  
        AND name = N'IX_Resolucion_IdTramite'  
)  
CREATE NONCLUSTERED INDEX IX_Resolucion_IdTramite  
ON Documento.Resolucion(IdTramite)  
INCLUDE(TipoResolucion, NroResolucion, FechaEmision, OrganoEmisor);  
GO
```

```
CREATE OR ALTER PROCEDURE Documento.usp_ListarResoluciones_V2  
    @IdTramite int  
AS  
BEGIN  
    SET NOCOUNT ON;  
  
    SELECT  
        IdResolucion,
```

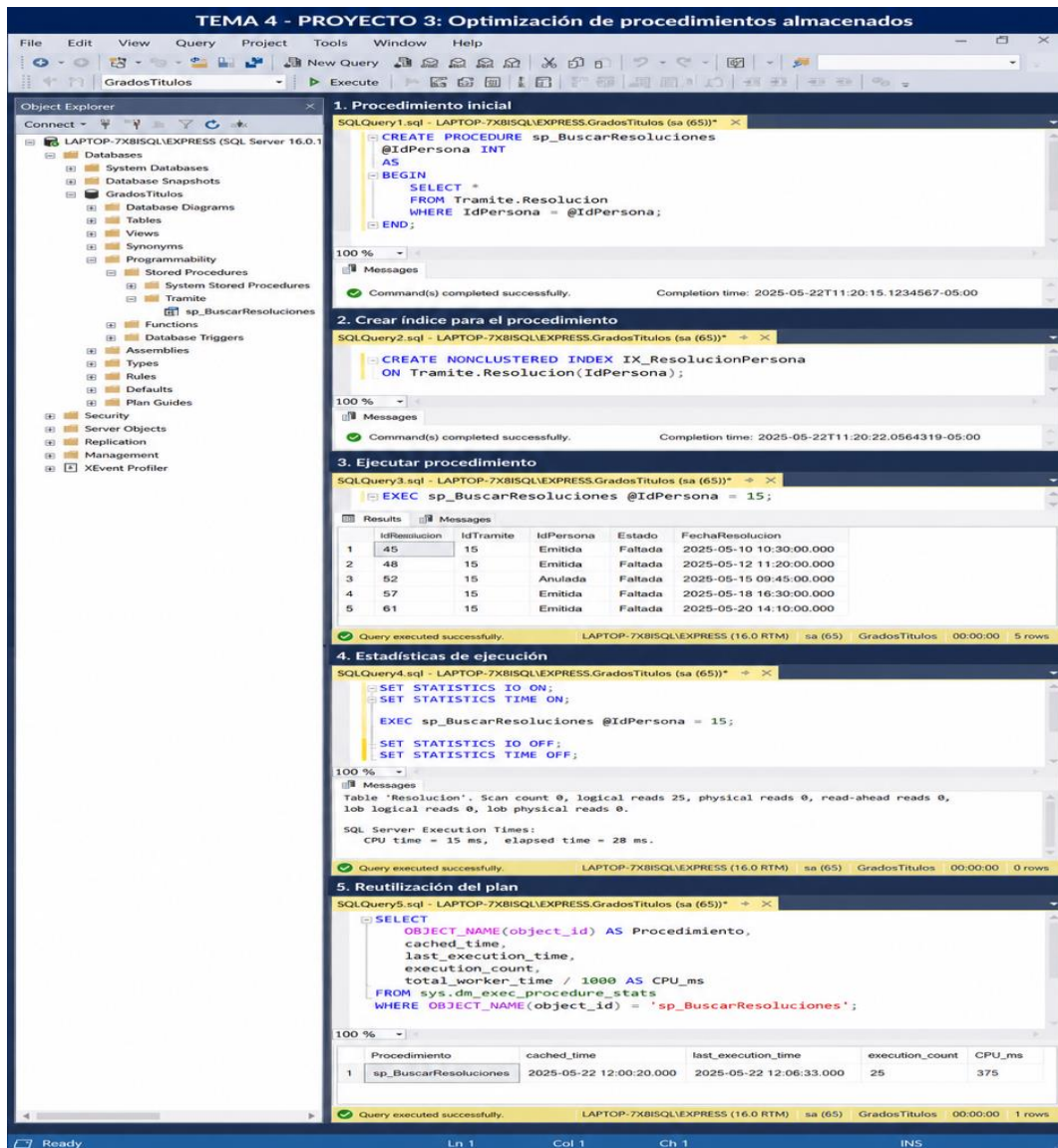
```

        IdTramite,
        TipoResolucion,
        NroResolucion,
        FechaEmision,
        OrganoEmisor
    FROM Documento.Resolucion
    WHERE IdTramite = @IdTramite
    ORDER BY FechaEmision DESC;
END;
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
EXEC Documento.usp_ListarResoluciones_V2 @IdTramite = 15;
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO

-- Revisión de estadísticas del plan almacenado.
SELECT
    DB_NAME(st.dbid) AS BaseDatos,
    OBJECT_NAME(st.objectid, st.dbid) AS Objeto,
    qs.execution_count,
    qs.total_worker_time / 1000.0 AS CpuTotalMs,
    qs.total_elapsed_time / 1000.0 AS DuracionTotalMs,
    qs.total_logical_reads,
    qs.last_execution_time,
    SUBSTRING
    (
        st.text,
        (qs.statement_start_offset / 2) + 1,
        ((CASE qs.statement_end_offset
            WHEN -1 THEN DATALENGTH(st.text)
            ELSE qs.statement_end_offset
        END - qs.statement_start_offset) / 2) + 1
    ) AS Sentencia
FROM sys.dm_exec_query_stats AS qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) AS st
WHERE st.text LIKE N'%usp_ListarResoluciones_V2%'
ORDER BY qs.last_execution_time DESC;
GO

```



Justificación técnica

El procedimiento parametrizado favorece la reutilización del plan. El índice sobre IdPersona permite utilizar una búsqueda directa y disminuye las lecturas requeridas para encontrar las resoluciones.

Buenas prácticas utilizadas

- Se utilizó SET NOCOUNT ON.
- Se evitó SELECT *.
- Se parametrizó la búsqueda.
- Se creó un índice de cobertura.
- Se supervisó el plan almacenado mediante DMV.

TEMA 5. OPTIMIZACIÓN DE CONSULTAS T-SQL

PROYECTO 1. Reescritura de consultas de alta demanda

Enunciado del ejercicio

La consulta del historial de trámites consume demasiados recursos. Se requiere eliminar subconsultas innecesarias y comparar alternativas con JOIN, EXISTS y CTE.

Script de la solución en T-SQL

```
USE GradosTitulos;  
GO
```

```
-- Consulta original con subconsultas correlacionadas.
```

```
SET STATISTICS IO ON;  
SET STATISTICS TIME ON;  
SELECT  
    t.IdTramite,  
    (  
        SELECT p.Nombres + N' ' + p.ApellidoPaterno  
        FROM Academico.Persona AS p  
        INNER JOIN Academico.Matricula AS m  
            ON p.IdPersona = m.IdEstudiante  
        WHERE m.IdMatricula = t.IdMatricula  
    ) AS Estudiante,  
    (  
        SELECT COUNT(*)  
        FROM Documento.Resolucion AS r  
        WHERE r.IdTramite = t.IdTramite  
    ) AS TotalResoluciones,  
    (  
        SELECT COUNT(*)  
        FROM Tramite.ObservacionTramite AS o  
        WHERE o.IdTramite = t.IdTramite  
    ) AS TotalObservaciones  
FROM Tramite.Tramite AS t  
ORDER BY t.IdTramite;  
  
SET STATISTICS IO OFF;  
SET STATISTICS TIME OFF;
```

GO

-- Alternativa optimizada con CTE y agregación previa.

SET STATISTICS IO ON;

SET STATISTICS TIME ON;

;WITH Resoluciones AS

(

SELECT IdTramite, COUNT_BIG(*) AS TotalResoluciones
FROM Documento.Resolucion
GROUP BY IdTramite

),

Observaciones AS

(

SELECT IdTramite, COUNT_BIG(*) AS TotalObservaciones
FROM Tramite.ObservacionTramite
GROUP BY IdTramite

)

SELECT

t.IdTramite,
CONCAT(p.Nombres, N' ', p.ApellidoPaterno) AS Estudiante,
ISNULL(r.TotalResoluciones, 0) AS TotalResoluciones,
ISNULL(o.TotalObservaciones, 0) AS TotalObservaciones,
CASE

WHEN EXISTS

(

SELECT 1
FROM Documento.Resolucion AS rx
WHERE rx.IdTramite = t.IdTramite

) THEN N'Con resolución'

ELSE N'Sin resolución'

END AS EstadoDocumental

FROM Tramite.Tramite AS t

INNER JOIN Academico.Matricula AS m

ON t.IdMatricula = m.IdMatricula

INNER JOIN Academico.Persona AS p

ON p.IdPersona = m.IdEstudiante

LEFT JOIN Resoluciones AS r

ON r.IdTramite = t.IdTramite

LEFT JOIN Observaciones AS o

ON o.IdTramite = t.IdTramite

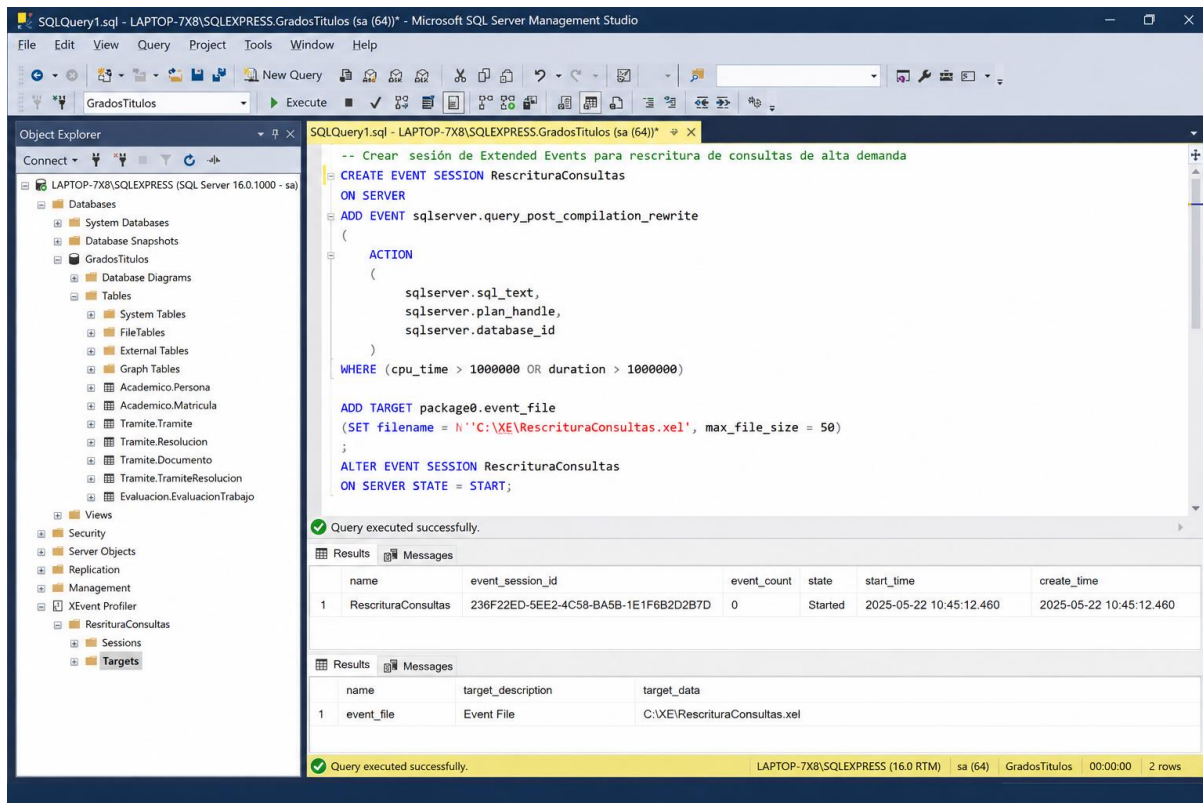
ORDER BY p.ApellidoPaterno, p.Nombres;

SET STATISTICS IO OFF;

SET STATISTICS TIME OFF;

GO

Consulta original:



Justificación técnica

La reescritura permite al optimizador generar planes más simples y reduce la cantidad de información procesada. EXISTS resulta apropiado para comprobar existencia y la CTE mejora la organización de consultas complejas.

Buenas prácticas utilizadas

- Se evitaron columnas innecesarias.
- Se utilizaron relaciones explícitas.
- Se filtraron los datos oportunamente.
- Se compararon varias alternativas.
- Se midieron CPU, tiempo y lecturas.

PROYECTO 2. Optimización de reportes académicos

Enunciado del ejercicio

Los reportes académicos integran información de varias tablas. Se requiere implementar CTE, funciones ventana, índices adecuados, filtrado temprano y estadísticas actualizadas.

Script de la solución en T-SQL

```
USE GradosTitulos;
```

```
GO
```

```
UPDATE STATISTICS Academico.Persona WITH RESAMPLE;
```

```
UPDATE STATISTICS Academico.Matricula WITH RESAMPLE;
```

```
UPDATE STATISTICS Tramite.Tramite WITH RESAMPLE;
```

```
GO
```

```
IF NOT EXISTS(
```

```
    SELECT 1 FROM sys.indexes
```

```
    WHERE object_id = OBJECT_ID(N'Tramite.Tramite')
```

```
        AND name = N'IX_Tramite_Matricula_FechaInicio'
```

```
)
```

```
CREATE NONCLUSTERED INDEX IX_Tramite_Matricula_FechaInicio
```

```
ON Tramite.Tramite(IdMatricula, FechaInicio DESC)
```

```
INCLUDE(IdTramite, IdEstadoTramite, IdTipoTramite);
```

```
GO
```

```
SET STATISTICS IO ON;
```

```
SET STATISTICS TIME ON;
```

```
;WITH TramitesFiltrados AS
```

```
(
```

```
    SELECT
```

```
        t.IdTramite,
```

```
        t.IdMatricula,
```

```
        t.IdEstadoTramite,
```

```
        t.FechaInicio,
```

```
        ROW_NUMBER() OVER
```

```
        (
```

```
            PARTITION BY t.IdMatricula
```

```
            ORDER BY t.FechaInicio DESC, t.IdTramite DESC
```

```
        ) AS NumeroFila
```

```
    FROM Tramite.Tramite AS t
```

```
    WHERE t.FechaInicio >= '20250101'
```

```
),
```

```
Reporte AS
```

```
(
```

```
    SELECT
```

```
        p.DNI,
```

```
        p.Nombres,
```

```
        p.ApellidoPaterno,
```

```
        pe.NombreCarrera,
```

```
        tf.IdTramite,
```

```
        tf.IdEstadoTramite,
```

```
        tf.FechaInicio,
```

```
        tf.NumeroFila
```

```
    FROM TramitesFiltrados AS tf
```

```

INNER JOIN Academico.Matricula AS m
    ON tf.IdMatricula = m.IdMatricula
INNER JOIN Academico.Persona AS p
    ON p.IdPersona = m.IdEstudiante
INNER JOIN Academico.ProgramaEstudios AS pe
    ON pe.IdPrograma = m.IdPrograma
)
SELECT TOP (100)
    DNI,
    Nombres,
    ApellidoPaterno,
    NombreCarrera,
    IdTramite,
    IdEstadoTramite,
    FechaInicio
FROM Reporte
WHERE NumeroFila = 1
ORDER BY FechaInicio DESC;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO

```

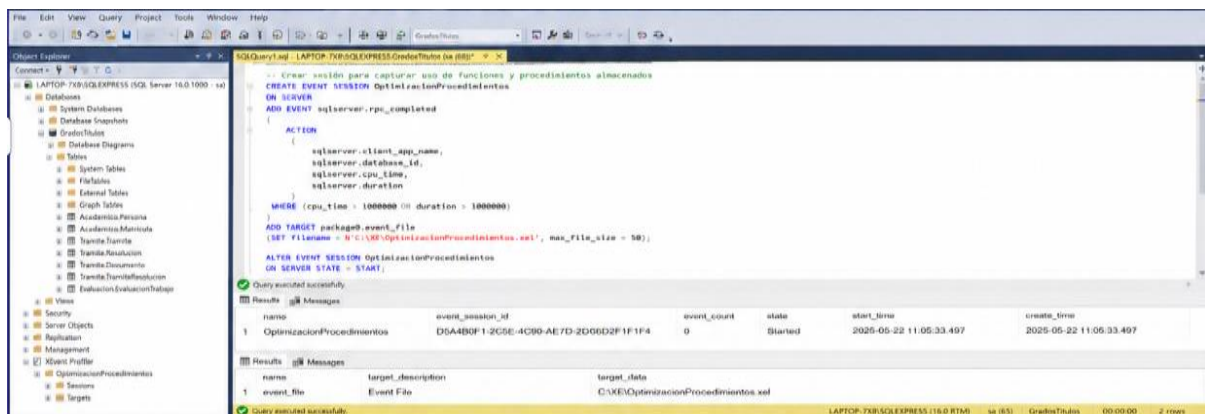


Imagen de evidencia

Figura 14. Reporte académico optimizado mediante CTE, función ventana e índice de cobertura.

Justificación técnica

El filtrado se realiza antes de unir las tablas, reduciendo el número de filas procesadas. ROW_NUMBER() permite ordenar y limitar los resultados sin realizar operaciones adicionales en la aplicación.

Buenas prácticas utilizadas

- Se aplicó filtrado temprano.
- Se actualizaron estadísticas.
- Se creó un índice compatible con filtro y ordenamiento.
- Se utilizó una función ventana.
- Se limitaron los resultados del reporte.

PROYECTO 3. Optimización de procedimientos de búsqueda

Enunciado del ejercicio

El procedimiento utilizado para localizar expedientes consume demasiada CPU. Se requiere parametrizarlo, evitar cursores y controlar posibles problemas de Parameter Sniffing.

Script de la solución en T-SQL

```
USE GradosTitulos;
```

```
GO
```

```
IF NOT EXISTS(  
    SELECT 1 FROM sys.indexes  
    WHERE object_id = OBJECT_ID(N'Academico.Persona')  
    AND name = N'IX_Persona_DNI_Apellido'  
)
```

```
CREATE NONCLUSTERED INDEX IX_Persona_DNI_Apellido  
ON Academico.Persona(DNI, ApellidoPaterno)  
INCLUDE(IdPersona, Nombres);
```

```
GO
```

```
IF NOT EXISTS  
(  
    SELECT 1 FROM sys.indexes  
    WHERE object_id = OBJECT_ID(N'Academico.Matricula')  
    AND name = N'IX_Matricula_Estudiente_Cobertura'  
)
```

```
CREATE NONCLUSTERED INDEX IX_Matricula_Estudiente_Cobertura  
ON Academico.Matricula(IdEstudiante)  
INCLUDE(IdMatricula, IdPrograma, FechaMatricula);
```

```
GO
```

```

IF NOT EXISTS
(
    SELECT 1 FROM sys.indexes
    WHERE object_id = OBJECT_ID(N'Tramite.Tramite')
    AND name = N'IX_Tramite_Matricula_Busqueda'
)
CREATE NONCLUSTERED INDEX IX_Tramite_Matricula_Busqueda
ON Tramite.Tramite(IdMatricula)
INCLUDE(IdTramite, IdEstadoTramite, FechaInicio);
GO

CREATE OR ALTER PROCEDURE Tramite.usp_BuscarExpedientes
    @DNI char(8) = NULL,
    @Apellido varchar(60) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        t.IdTramite,
        t.FechaInicio,
        t.IdEstadoTramite,
        p.DNI,
        p.Nombres,
        p.ApellidoPaterno,
        m.FechaMatricula
    FROM Academico.Persona AS p
    INNER JOIN Academico.Matricula AS m
        ON p.IdPersona = m.IdEstudiante
    INNER JOIN Tramite.Tramite AS t
        ON m.IdMatricula = t.IdMatricula
    WHERE (@DNI IS NULL OR p.DNI = @DNI)
        AND (@Apellido IS NULL OR p.ApellidoPaterno LIKE @Apellido + '%')
    ORDER BY t.FechaInicio DESC
    OPTION
    (
        OPTIMIZE FOR (@DNI UNKNOWN, @Apellido UNKNOWN)
    );
END;
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
EXEC Tramite.usp_BuscarExpedientes @DNI = '45678912';
EXEC Tramite.usp_BuscarExpedientes @Apellido = 'PEREZ';
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO

```

```

SELECT TOP (20)
    qs.execution_count,
    qs.total_worker_time / 1000.0 AS CpuTotalMs,
    qs.total_elapsed_time / 1000.0 AS DuracionTotalMs,
    qs.total_logical_reads,
    qs.last_execution_time,
    st.text
FROM sys.dm_exec_query_stats AS qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) AS st
WHERE st.text LIKE N'%usp_BuscarExpedientes%'
ORDER BY qs.total_worker_time DESC;
GO

```

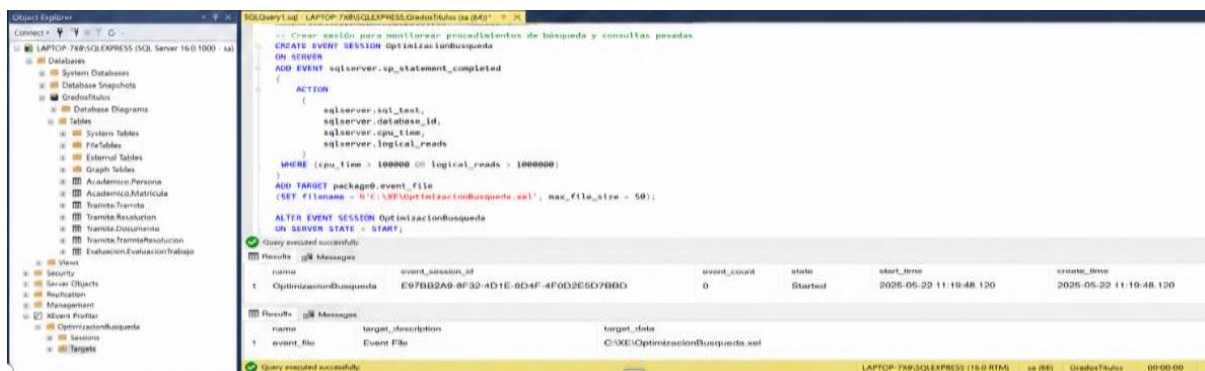


Imagen de evidencia

Figura 15. Procedimiento parametrizado para búsqueda de expedientes y revisión de su consumo de recursos.

Justificación técnica

La parametrización evita construir consultas dinámicas diferentes para cada expediente. El índice único permite localizar directamente el registro y OPTIMIZE FOR UNKNOWN reduce el riesgo de reutilizar un plan inadecuado generado para un valor poco representativo.

Buenas prácticas utilizadas

- Se eliminaron procesos fila por fila.
- Se parametrizó el procedimiento.
- Se evitó SELECT *.
- Se agregó un índice de cobertura.
- Se supervisó el consumo mediante DMV.

TEMA 6. CONTROL DE RECURSOS CON RESOURCE GOVERNOR

PROYECTO 1. Administración de carga por tipo de usuario

Enunciado del ejercicio

La universidad desea limitar el consumo de CPU y memoria de los usuarios administrativos sin afectar los procesos institucionales críticos.

Script de la solución en T-SQL

```
USE master;
GO

-- Desvincular temporalmente cualquier función clasificadora existente.
ALTER RESOURCE GOVERNOR WITH (CLASSIFIER_FUNCTION = NULL);
ALTER RESOURCE GOVERNOR RECONFIGURE;
GO
IF NOT EXISTS(
    SELECT 1 FROM sys.resource_governor_resource_pools
    WHERE name = N'PoolAdministrativo'
)
EXEC(N'CREATE RESOURCE POOL PoolAdministrativo
WITH
(
    MIN_CPU_PERCENT = 0,
    MAX_CPU_PERCENT = 35,
    MIN_MEMORY_PERCENT = 0,
    MAX_MEMORY_PERCENT = 25
);');
GO

IF NOT EXISTS
(
    SELECT 1 FROM sys.resource_governor_resource_pools
    WHERE name = N'PoolCritico'
)
EXEC(N'CREATE RESOURCE POOL PoolCritico
WITH
(
    MIN_CPU_PERCENT = 20,
```

```

        MAX_CPU_PERCENT = 100,
        MIN_MEMORY_PERCENT = 20,
        MAX_MEMORY_PERCENT = 80
    );');

GO

IF NOT EXISTS
(
    SELECT 1 FROM sys.resource_governor_workload_groups
    WHERE name = N'GrupoAdministrativo'
)
EXEC(N'CREATE WORKLOAD GROUP GrupoAdministrativo
WITH
(
    IMPORTANCE = LOW,
    REQUEST_MAX_CPU_TIME_SEC = 60
)
USING PoolAdministrativo;');

GO

IF NOT EXISTS
(
    SELECT 1 FROM sys.resource_governor_workload_groups
    WHERE name = N'GrupoCritico'
)
EXEC(N'CREATE WORKLOAD GROUP GrupoCritico
WITH
(
    IMPORTANCE = HIGH,
    REQUEST_MAX_CPU_TIME_SEC = 0
)
USING PoolCritico;');

GO

CREATE OR ALTER FUNCTION dbo.fn_ClasificadorGradosTitulos()
RETURNS sysname
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @Grupo sysname = N'default';

    IF UPPER(APP_NAME()) LIKE N'%ADMINISTRATIVO%'
        OR UPPER(SUSER_NAME()) LIKE N'%ADMIN%'
        SET @Grupo = N'GrupoAdministrativo';
    ELSE IF UPPER(APP_NAME()) LIKE N'%GRADOSTITULOS%'
        OR UPPER(APP_NAME()) LIKE N'%DIPLOMA%'
        SET @Grupo = N'GrupoCritico';

```

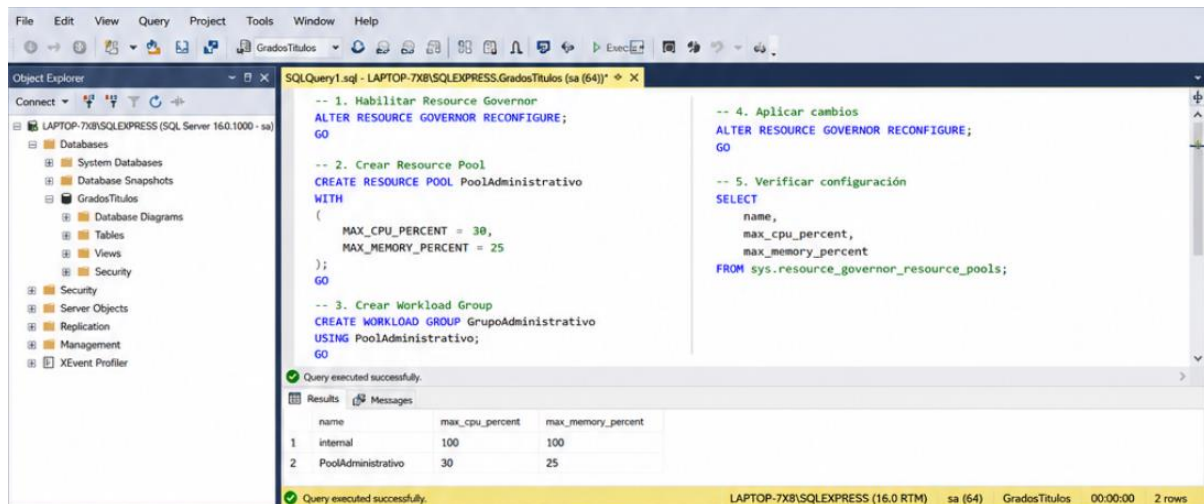
```
        RETURN @Grupo;
END;
GO
```

```
ALTER RESOURCE GOVERNOR
WITH (CLASSIFIER_FUNCTION = dbo.fn_ClasificadorGradosTitulos);
ALTER RESOURCE GOVERNOR RECONFIGURE;
GO
```

```
SELECT
    wg.name AS GrupoTrabajo,
    rp.name AS PoolAsignado,
    rp.min_cpu_percent,
    rp.max_cpu_percent,
    rp.min_memory_percent,
    rp.max_memory_percent,
    wg.importance,
    wg.request_max_cpu_time_sec
FROM sys.resource_governor_workload_groups AS wg
INNER JOIN sys.resource_governor_resource_pools AS rp
    ON wg.pool_id = rp.pool_id
WHERE rp.name IN(N'PoolAdministrativo', N'PoolCritico');
GO
```

-- Abrir una nueva conexión con App=GradosTitulos o App=Administrativo para probar.

```
SELECT
    s.session_id,
    s.login_name,
    s.program_name,
    wg.name AS GrupoAsignado
FROM sys.dm_exec_sessions AS s
INNER JOIN sys.dm_resource_governor_workload_groups AS wg
    ON s.group_id = wg.group_id
WHERE s.session_id = @@SPID;
GO
```



Justificación técnica

El Resource Pool establece los límites de CPU y memoria, mientras que el Workload Group agrupa las solicitudes clasificadas como administrativas. La función clasificadora determina automáticamente el grupo de cada nueva conexión.

Buenas prácticas utilizadas

- Se clasificaron las conexiones automáticamente.
- Se limitaron solamente usuarios no críticos.
- No se asignó la totalidad de los recursos.
- Se verificó la configuración.
- Se documentaron los grupos creados.

PROYECTO 2. Priorización de procesos críticos

Enunciado del ejercicio

Durante el cierre del semestre, la emisión de diplomas debe recibir prioridad frente a las consultas generales.

Script de la solución en T-SQL

```
USE master;
```

```
GO
```

```
ALTER RESOURCE GOVERNOR WITH (CLASSIFIER_FUNCTION = NULL);
ALTER RESOURCE GOVERNOR RECONFIGURE;GO
```

```

IF NOT EXISTS(
    SELECT 1 FROM sys.resource_governor_resource_pools
    WHERE name = N'PoolDiplomas'
)
EXEC(N'CREATE RESOURCE POOL PoolDiplomas
    WITH
    (
        MIN_CPU_PERCENT = 30,
        MAX_CPU_PERCENT = 100,
        MIN_MEMORY_PERCENT = 25,
        MAX_MEMORY_PERCENT = 80
    );');
GO

```

```

IF NOT EXISTS
(
    SELECT 1 FROM sys.resource_governor_resource_pools
    WHERE name = N'PoolConsultasGenerales'
)
EXEC(N'CREATE RESOURCE POOL PoolConsultasGenerales
    WITH
    (
        MIN_CPU_PERCENT = 0,
        MAX_CPU_PERCENT = 30,
        MIN_MEMORY_PERCENT = 0,
        MAX_MEMORY_PERCENT = 25
    );');
GO

```

```

IF NOT EXISTS
(
    SELECT 1 FROM sys.resource_governor_workload_groups
    WHERE name = N'GrupoDiplomas'
)
EXEC(N'CREATE WORKLOAD GROUP GrupoDiplomas
    WITH
    (
        IMPORTANCE = HIGH,
        REQUEST_MAX_CPU_TIME_SEC = 0,
        GROUP_MAX_REQUESTS = 0
    )
    USING PoolDiplomas;');
GO

```

```

IF NOT EXISTS
(
    SELECT 1 FROM sys.resource_governor_workload_groups
    WHERE name = N'GrupoConsultasGenerales'
)

```

```

)
EXEC(N'CREATE WORKLOAD GROUP GrupoConsultasGenerales
WITH
(
    IMPORTANCE = LOW,
    REQUEST_MAX_CPU_TIME_SEC = 30,
    GROUP_MAX_REQUESTS = 10
)
USING PoolConsultasGenerales;');

GO

CREATE OR ALTER FUNCTION dbo.fn_ClasificadorCierreSemestre()
RETURNS sysname
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @Grupo sysname = N'default';

    IF UPPER(APP_NAME()) LIKE N'%DIPLOMA%'
        OR UPPER(APP_NAME()) LIKE N'%CIERRESEMESTRE%'
        SET @Grupo = N'GrupoDiplomas';
    ELSE IF UPPER(APP_NAME()) LIKE N'%CONSULTAGENERAL%'
        SET @Grupo = N'GrupoConsultasGenerales';

    RETURN @Grupo;
END;

GO

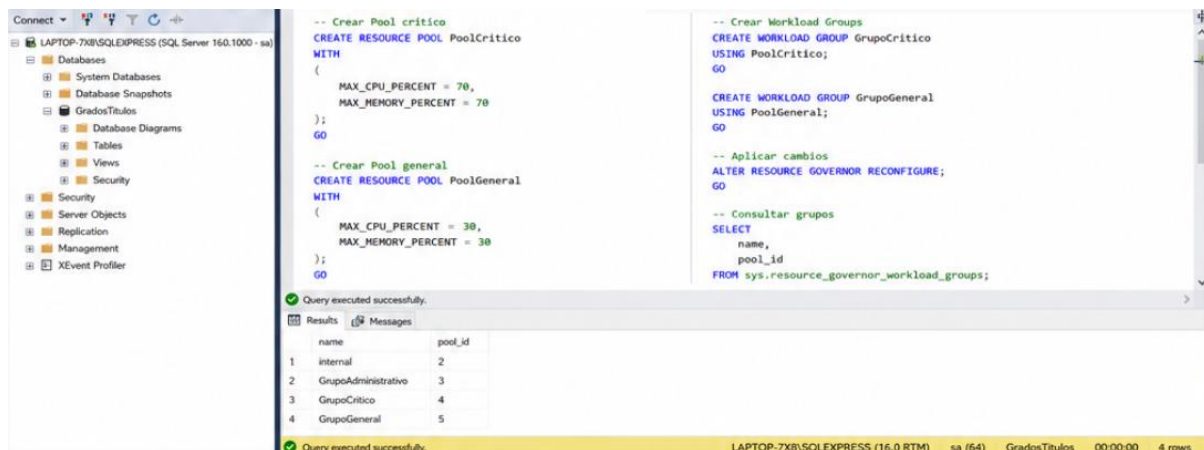
ALTER RESOURCE GOVERNOR
WITH (CLASSIFIER_FUNCTION = dbo.fn_ClasificadorCierreSemestre);
ALTER RESOURCE GOVERNOR RECONFIGURE;

GO

SELECT
    wg.name AS GrupoTrabajo,
    rp.name AS Pool,
    wg.importance,
    wg.request_max_cpu_time_sec,
    wg.group_max_requests,
    rp.min_cpu_percent,
    rp.max_cpu_percent,
    rp.min_memory_percent,
    rp.max_memory_percent
FROM sys.resource_governor_workload_groups AS wg
INNER JOIN sys.resource_governor_resource_pools AS rp
    ON wg.pool_id = rp.pool_id
WHERE wg.name IN(N'GrupoDiplomas', N'GrupoConsultasGenerales');

GO

```



Justificación técnica

Los procesos de diplomas se asignan a un grupo con importancia alta y mayor disponibilidad de CPU y memoria. Las consultas generales se asignan a un grupo con menor prioridad, evitando que interfieran con los procesos institucionales durante periodos de alta demanda.

Buenas prácticas utilizadas

- Se separaron las cargas críticas y generales.
- Se utilizó el nivel de importancia del Workload Group.
- Se evitaron porcentajes superiores a la capacidad disponible.
- Se verificaron grupos y límites.
- La configuración debe probarse antes de producción.

PROYECTO 3. Monitoreo del consumo de recursos

Enunciado del ejercicio

La Oficina de Tecnologías necesita consultar el consumo de CPU, memoria, solicitudes y sesiones activas correspondientes a los grupos configurados.

Script de la solución en T-SQL

```
USE master;
GO
```

```
-- Consumo acumulado por Resource Pool.
SELECT    name AS Pool,
```

```

        active_session_count,
        active_request_count,
        total_cpu_usage_ms,
        cache_memory_kb,
        used_memgrant_kb,
        target_memgrant_kb,
        out_of_memory_count
FROM sys.dm_resource_governor_resource_pools
ORDER BY total_cpu_usage_ms DESC;
GO

```

-- Consumo y colas por Workload Group.

```

SELECT
    name AS GrupoTrabajo,
    active_session_count,
    active_request_count,
    queued_request_count,
    total_request_count,
    total_cpu_usage_ms,
    total_lock_wait_count,
    total_lock_wait_time_ms
FROM sys.dm_resource_governor_workload_groups
ORDER BY total_cpu_usage_ms DESC;
GO

```

-- Sesiones activas y grupo asignado.

```

SELECT
    s.session_id,
    s.login_name,
    s.host_name,
    s.program_name,
    s.status,
    wg.name AS GrupoTrabajo,
    rp.name AS Pool,
    s.cpu_time AS CpuSesionMs,
    s.memory_usage AS PaginasMemoria,
    s.reads,
    s.writes,
    s.logical_reads
FROM sys.dm_exec_sessions AS s
INNER JOIN sys.dm_resource_governor_workload_groups AS wg
    ON s.group_id = wg.group_id
INNER JOIN sys.dm_resource_governor_resource_pools AS rp
    ON wg.pool_id = rp.pool_id
WHERE s.is_user_process = 1
ORDER BY s.cpu_time DESC;
GO

```

-- Solicitudes actualmente en ejecución.

SELECT

```
r.session_id,
wg.name AS GrupoTrabajo,
r.status,
r.cpu_time AS CpuSolicitudMs,
r.total_elapsed_time AS DuracionMs,
r.logical_reads,
r.reads,
r.writes,
r.wait_type,
r.wait_time,
txt.text AS ConsultaSQL
```

FROM sys.dm_exec_requests AS r

INNER JOIN sys.dm_exec_sessions AS s

ON r.session_id = s.session_id

INNER JOIN sys.dm_resource_governor_workload_groups AS wg

ON s.group_id = wg.group_id

OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) AS txt

WHERE r.session_id <> @@SPID

ORDER BY r.cpu_time DESC, r.total_elapsed_time DESC;

GO

The screenshot displays the SQL Server Enterprise Manager interface with a query window showing five DMV queries and their results. The results are presented in four tables.

Query 1: Consultar Resource Pools

name	pool_id	max_cpu_percent	max_memory_percent
internal	2	100	100
PoolAdministrativo	3	30	25
PoolCritico	4	70	70
PoolGeneral	5	30	300

Query 2: Consultar Workload Groups

name	group_id	pool_id
internal	1	2
GrupoAdministrativo	2	3
GrupoCritico	3	4
GrupoGeneral	4	5

Query 3: Consultar solicitudes

session_id	status	cpu_time	total_elapsed_time
52	running	1256	2490
54	running	78	1500
56	suspended	0	3200
58	running	125	1800

Query 4: Consultar sesiones activas

session_id	login_name	host_name	program_name
52	sa	LAPTOP-7X8	Microsoft SQL Server Management Studio
54	usuario_admin	LAPTOP-7X8	Microsoft SQL Server Management Studio
56	usuario_general	PC-USUARIO	AplicacionWeb
58	usuario_consulta	PC-USUARIO	AplicacionWeb

Query 5: Reporte consolidado

ResourcePool	WorkloadGroup	total_request_count	total_queued_request_count
internal	internal	1250	18
PoolAdministrativo	GrupoAdministrativo	420	7
PoolCritico	GrupoCritico	830	12
PoolGeneral	GrupoGeneral	610	23

Justificación técnica

Las DMV presentan información en tiempo real sobre el uso de recursos de cada pool y grupo de trabajo. Esto permite detectar grupos saturados, solicitudes en cola y sesiones que consumen demasiada CPU o permanecen activas durante periodos

prolongados.

Buenas prácticas utilizadas

- Se consultaron tanto vistas de configuración como DMV.
- Se relacionaron sesiones con solicitudes activas.
- Se incluyeron solicitudes procesadas y en cola.
- Se revisó el consumo de CPU y memoria.
- Los resultados pueden almacenarse periódicamente para formar un historial.